

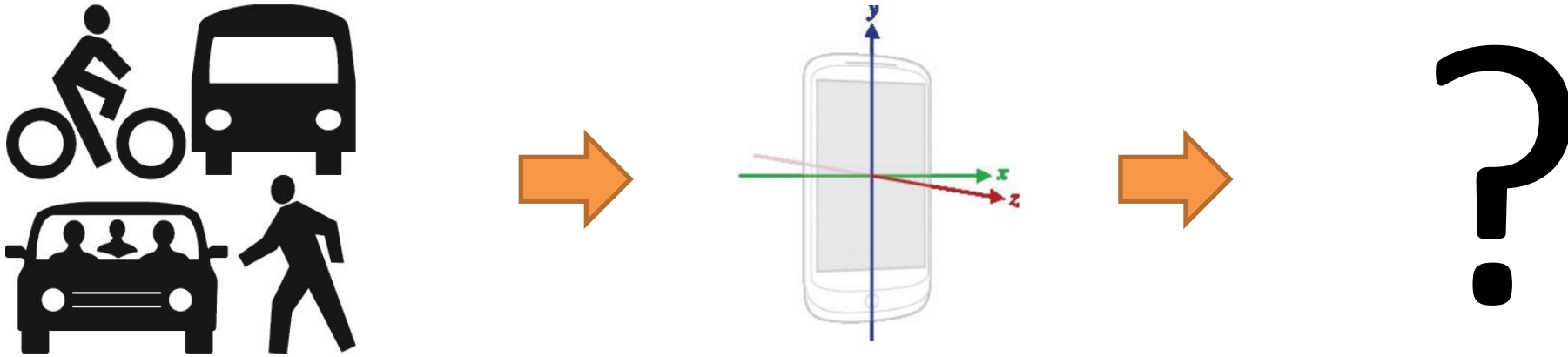
Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación



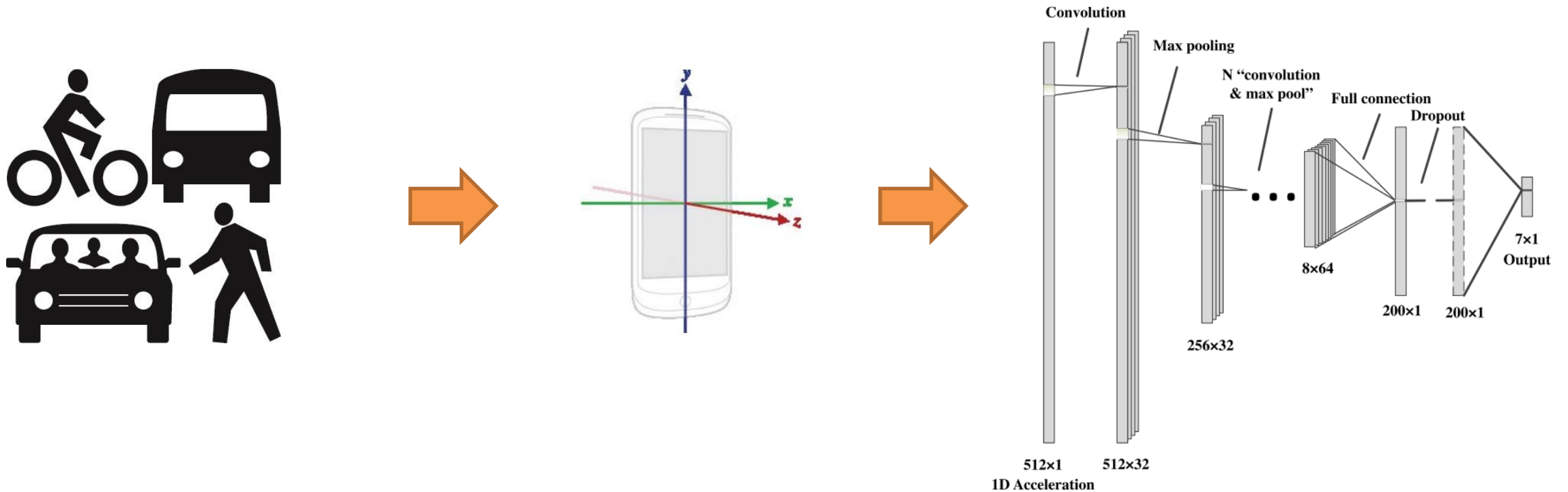
Deep Learning

Redes Neuronales Recurrentes

¿Cómo podríamos procesar una secuencia de datos para estimar el modo de transporte, con las redes neuronales que hemos visto hasta ahora?



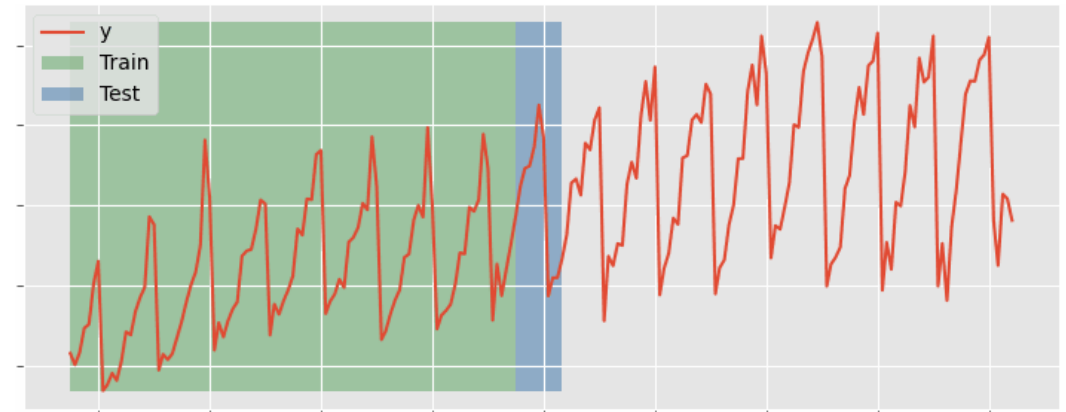
¿Cómo podríamos procesar una secuencia de datos para estimar el modo de transporte, con la redes neuronales que hemos visto hasta ahora?



¿Qué limitación tiene este enfoque?

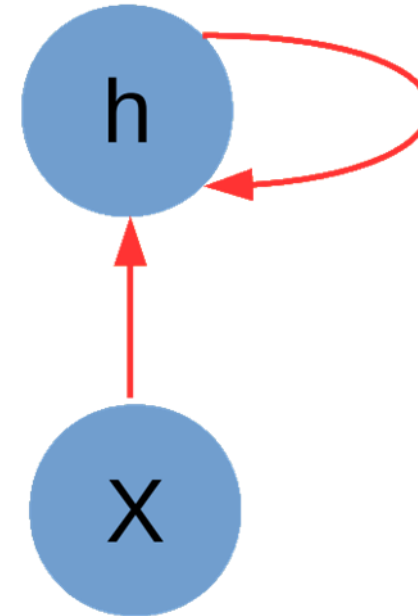
¿Qué técnicas veremos en esta segunda parte del curso para enfrentar estos problemas?

- Recurrencia para modelar el estado de una secuencia: **RNN, LSTM, GRU**.
- Modelos de lenguaje: **word2vec, seq2seq, seq2seq con atención**.
- Foundation Modelos: **Transformer, BERT, GPT y más**.

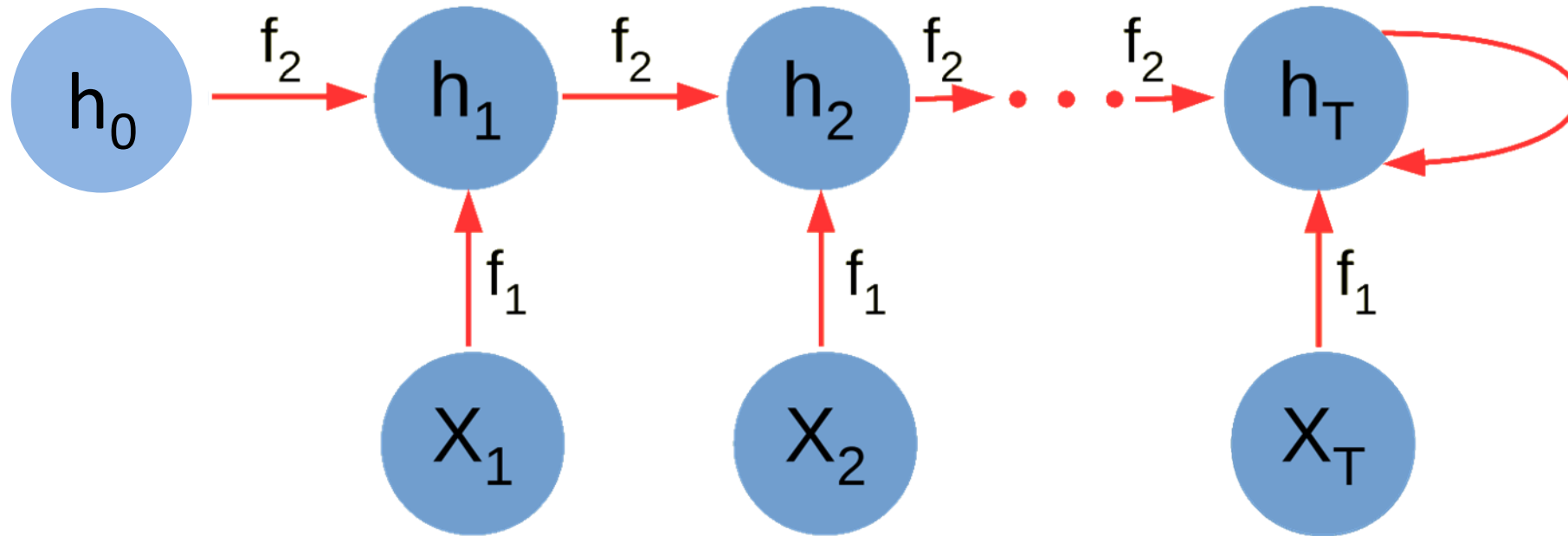


El primer enfoque que veremos serán las **Redes Neuronales Recurrentes** (RNN), diseñadas para procesar secuencias

- Propuestas en la década de los 80.
- Trabajan de manera secuencial, procesando uno a uno los elementos de la secuencia/serie.
- A diferencia de métodos tradicionales y de otros tipos de redes, las RNN mantienen un **estado**, es decir, “**tienen memoria**”.
- Cómo se maneja esta memoria es lo que caracteriza a los distintos tipos de redes recurrentes existentes (RNN, LSTM, GRU).

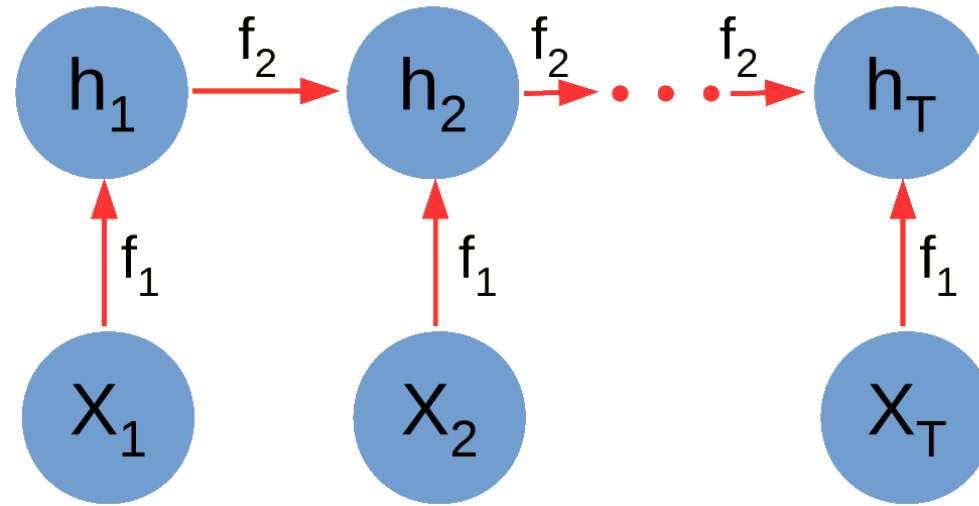


Expandamos este diagrama, para ver explícitamente el funcionamiento de una RNN en el tiempo (generando así el **grafo de cómputo asociado**)



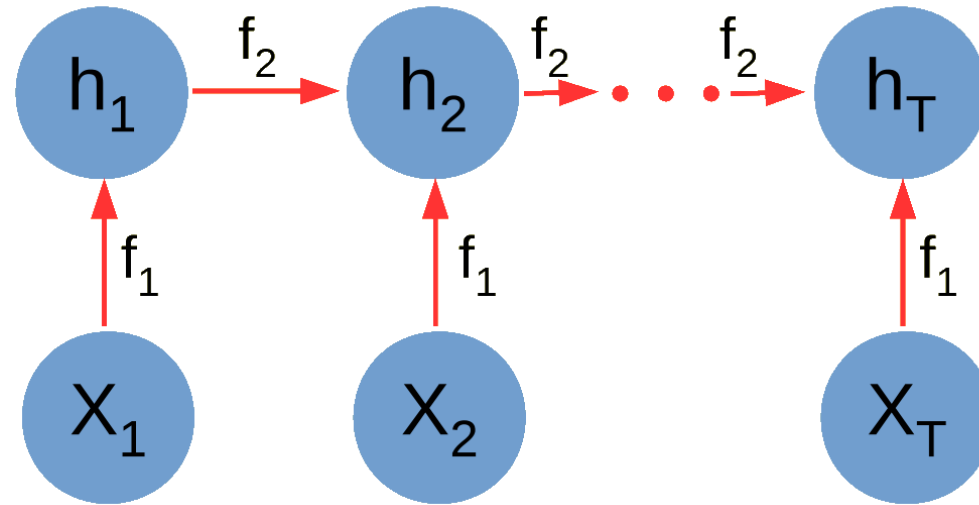
$$h_t = g(f_1(x_t), f_2(h_{t-1}))$$

Formalicemos un poco lo anterior



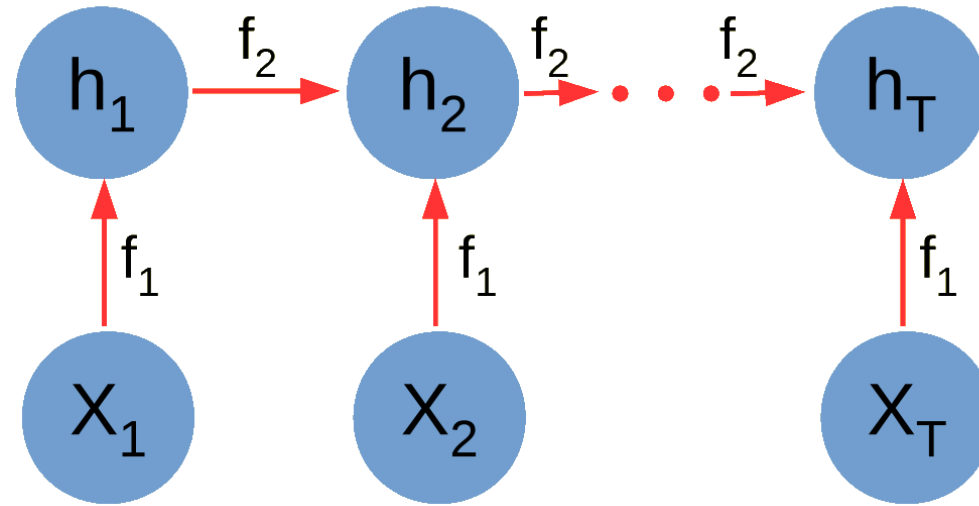
- x_t : vector de entrada que codifica la secuencia en el paso t
- h_t : vector de estado oculto (latente/interno) en el paso t , que codifica la **historia relevante** de la entrada hasta ese momento
- f_1 y f_2 : funciones paramétricas **entrenables** (aquí está la clave)
- t : paso de la secuencia, usualmente tiene significado temporal, pero puede representar cualquier relación de orden (espacio, ranking, etc).

La configuración típica de una RNN para g , f_1 y f_2 es con sigmoide y funciones lineales combinadas aditivamente



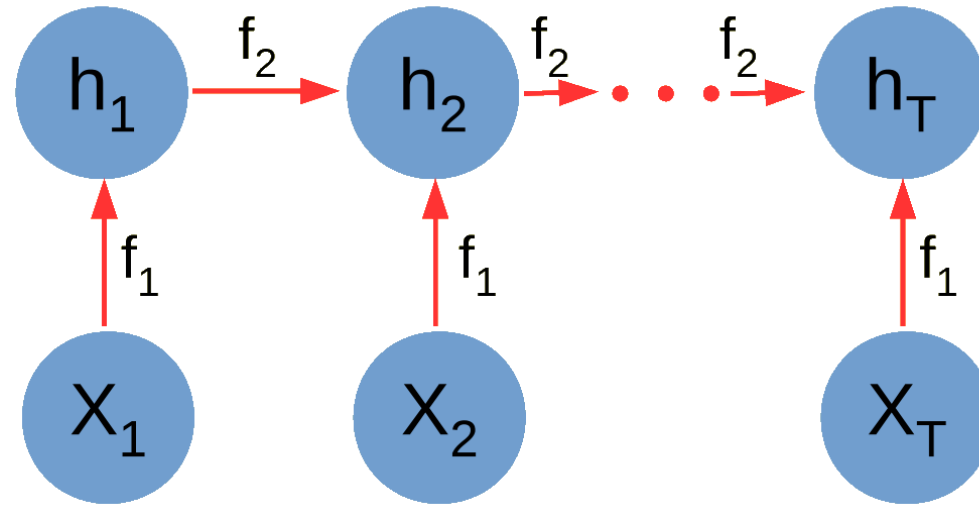
$$h_t = g(f_1(x_t), f_2(h_{t-1}))$$

La configuración típica de una RNN para g , f_1 y f_2 es con sigmoide y funciones lineales combinadas aditivamente



$$h_t = \sigma(W_{hh} h_{t-1} + W_{xh} x_t)$$

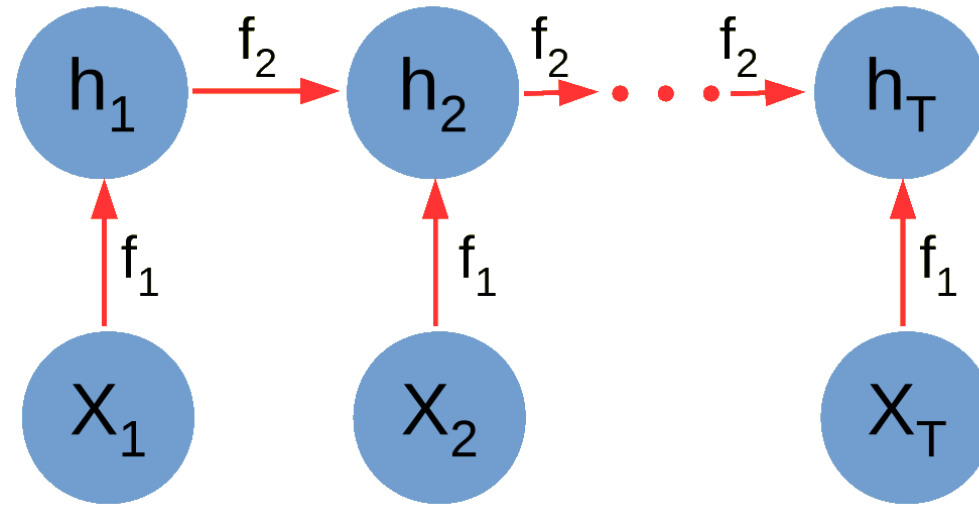
La configuración típica de una RNN para g , f_1 y f_2 es con sigmoide y funciones lineales combinadas aditivamente



$$h_t = \sigma(W_{hh} h_{t-1} + W_{xh} x_t)$$

- El tamaño del grafo depende del largo de la secuencia, pero ¿qué pasa con el número de parámetros?
- Al utilizar una capa recurrente, podemos capturar dependencias de longitud arbitraria en los datos, sin la obligación de aumentar la complejidad del modelo (no necesitamos filtros más grandes).

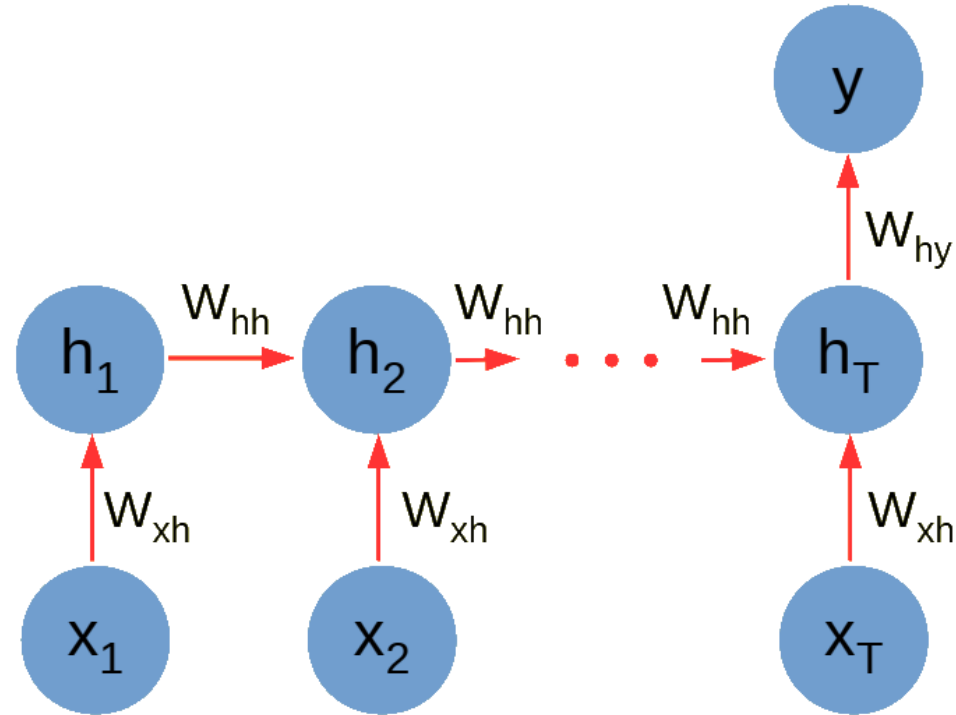
Las redes recurrentes como elementos de memoria



$$h_t = \sigma(W_{hh} h_{t-1} + W_{xh} x_t)$$

- Las RNN **aprenden** a usar el estado oculto h_t como una **feature** que representa lo relevante de las entradas hasta el paso t , para la tarea a resolver.
- Esto permite capturar la estructura secuencial/temporal de la entrada (análogamente a como las convoluciones capturan la estructura espacial en una CNN).

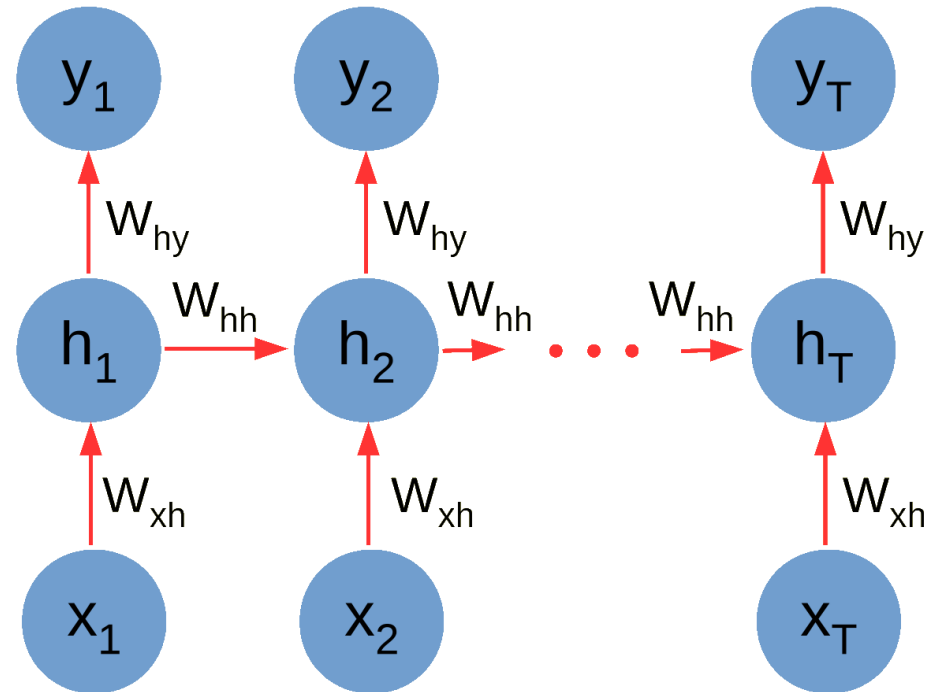
Para incorporar supervisión, seguimos usando las mismas ideas



$$h_t = \sigma(W_{hh} h_{t-1} + W_{xh} x_t)$$

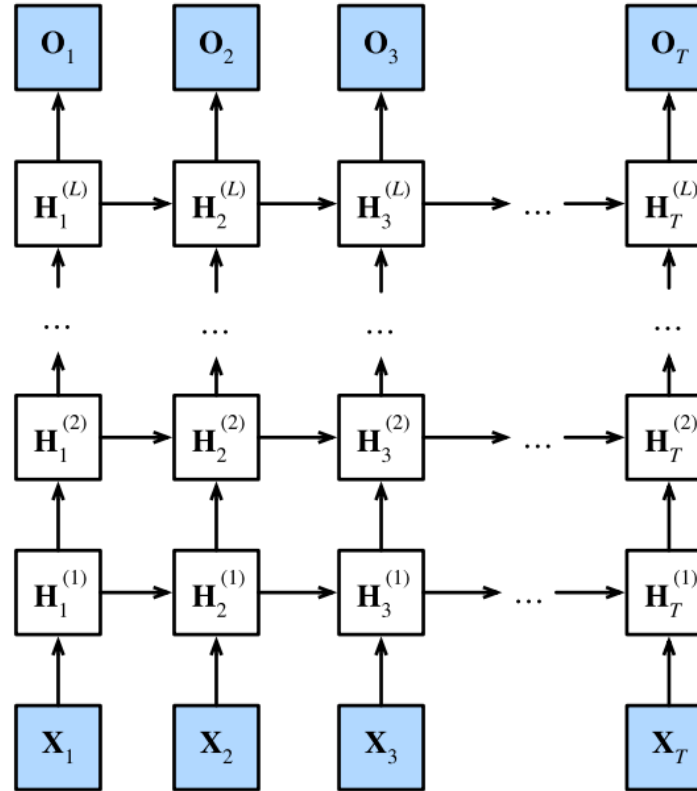
$$y = \sigma(W_{hy} h_T)$$

La salida de una RNN puede adecuarse a la tarea de turno



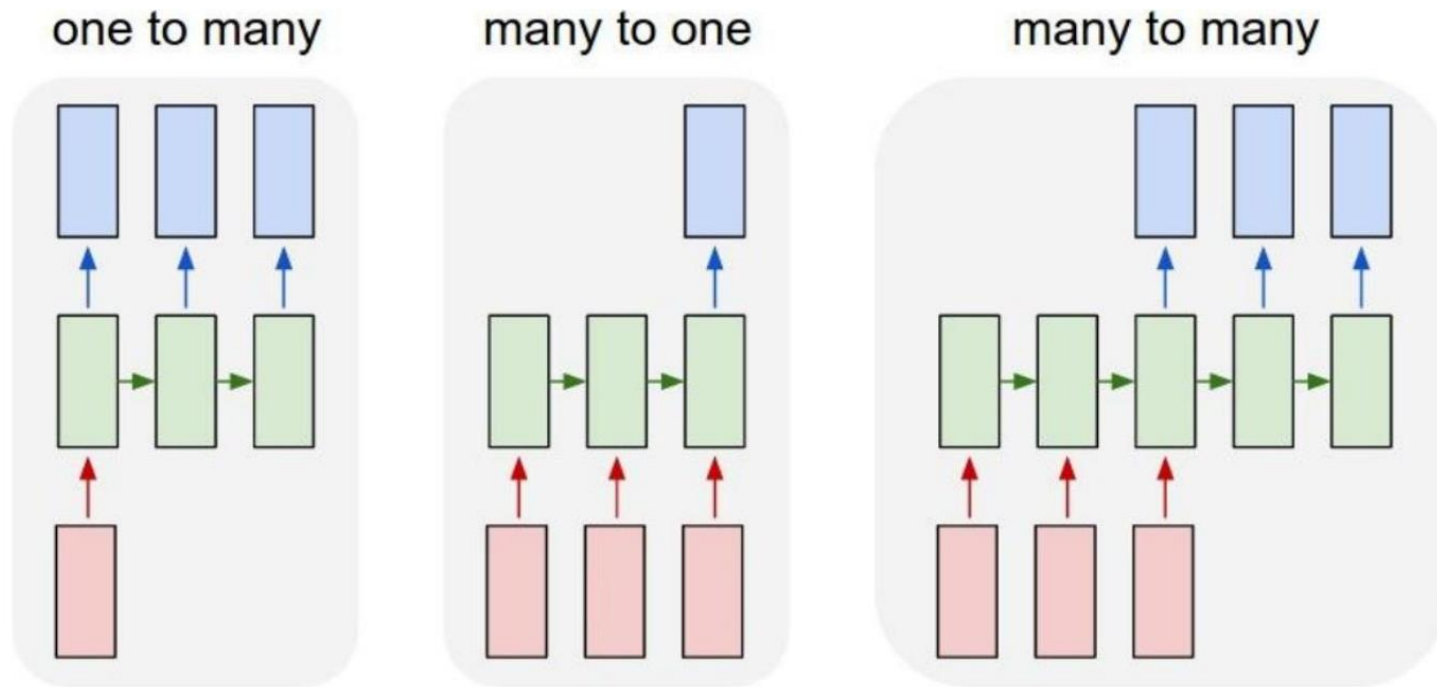
En esta configuración, la RNN genera una salida por cada valor de la secuencia de entrada.

La cantidad de capas de una RNN también puede adecuarse a las necesidades



RNN con múltiples capas requieren el paso de la secuencia completa de estados ocultos entre capas

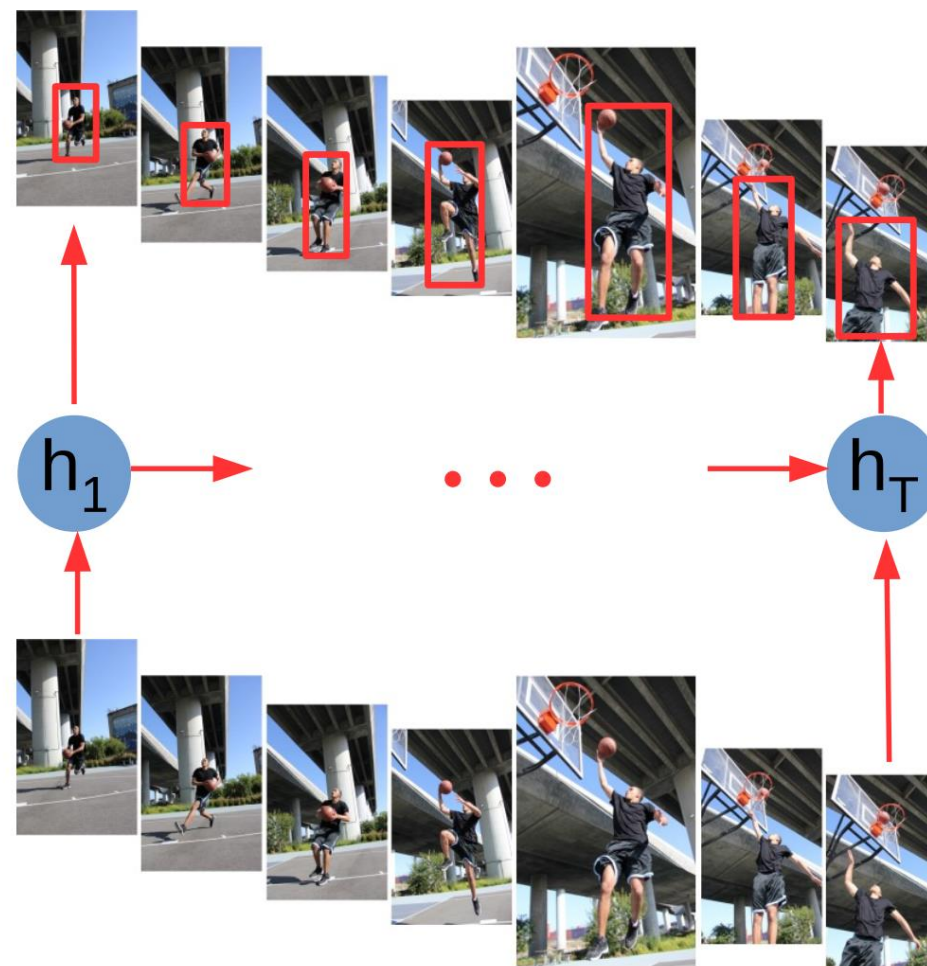
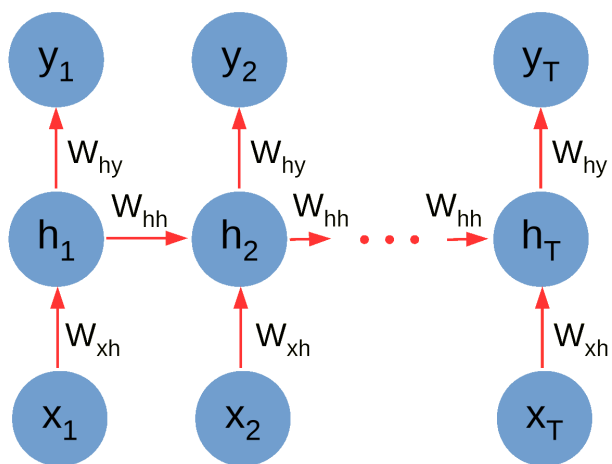
En la práctica, la estructura completa de una RNN puede adecuarse a las necesidades



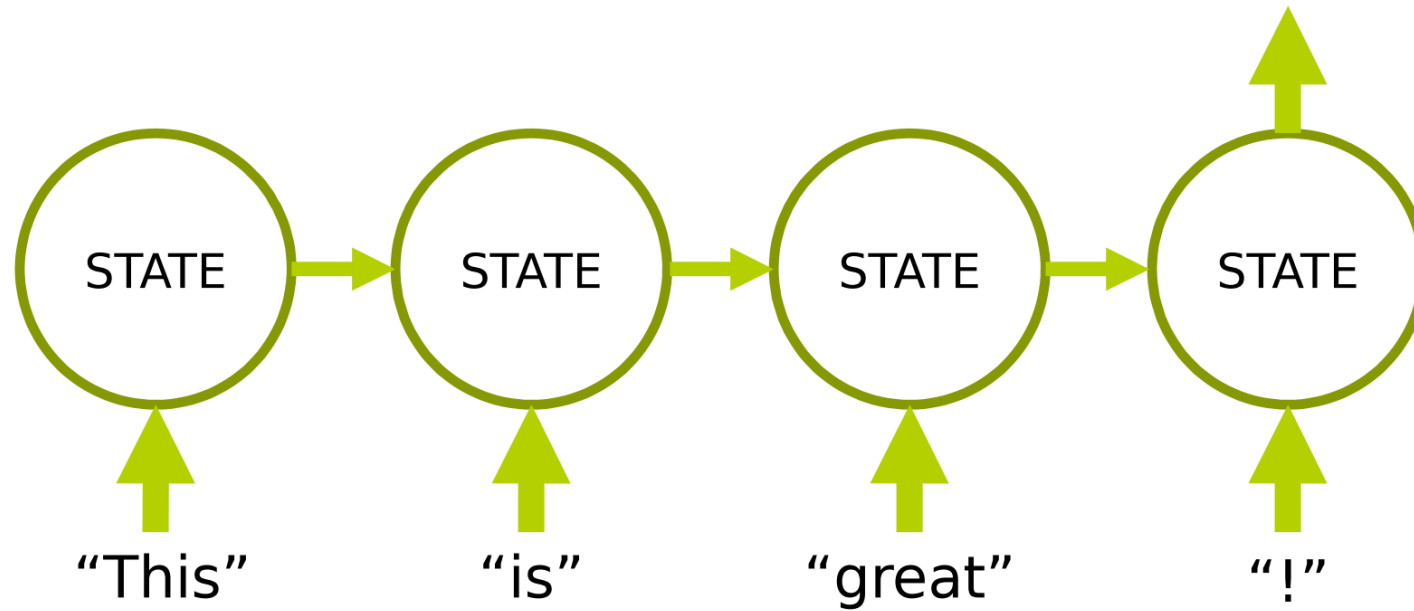
Vamos a Colab...



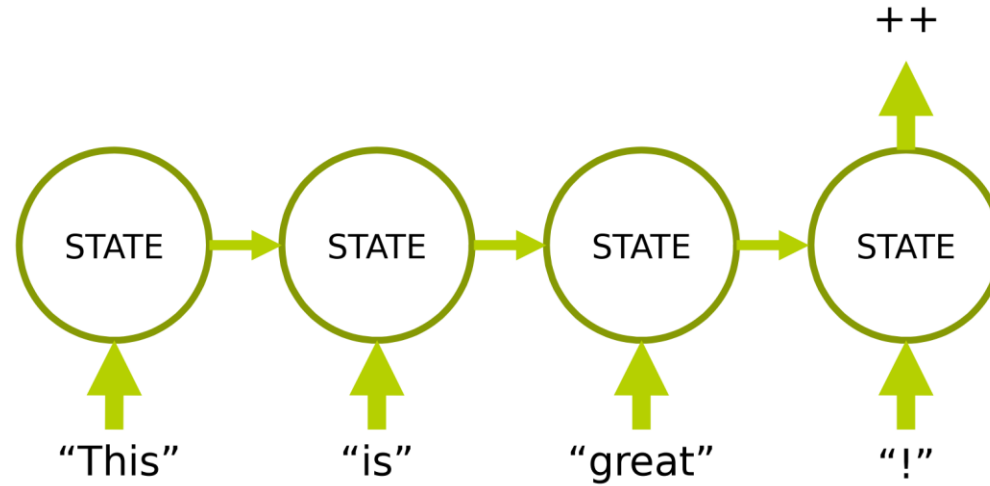
Conceptualicemos posibles aplicaciones de las RNN: seguimiento en videos



Conceptualicemos posibles aplicaciones de las RNN: predicción de sentimiento

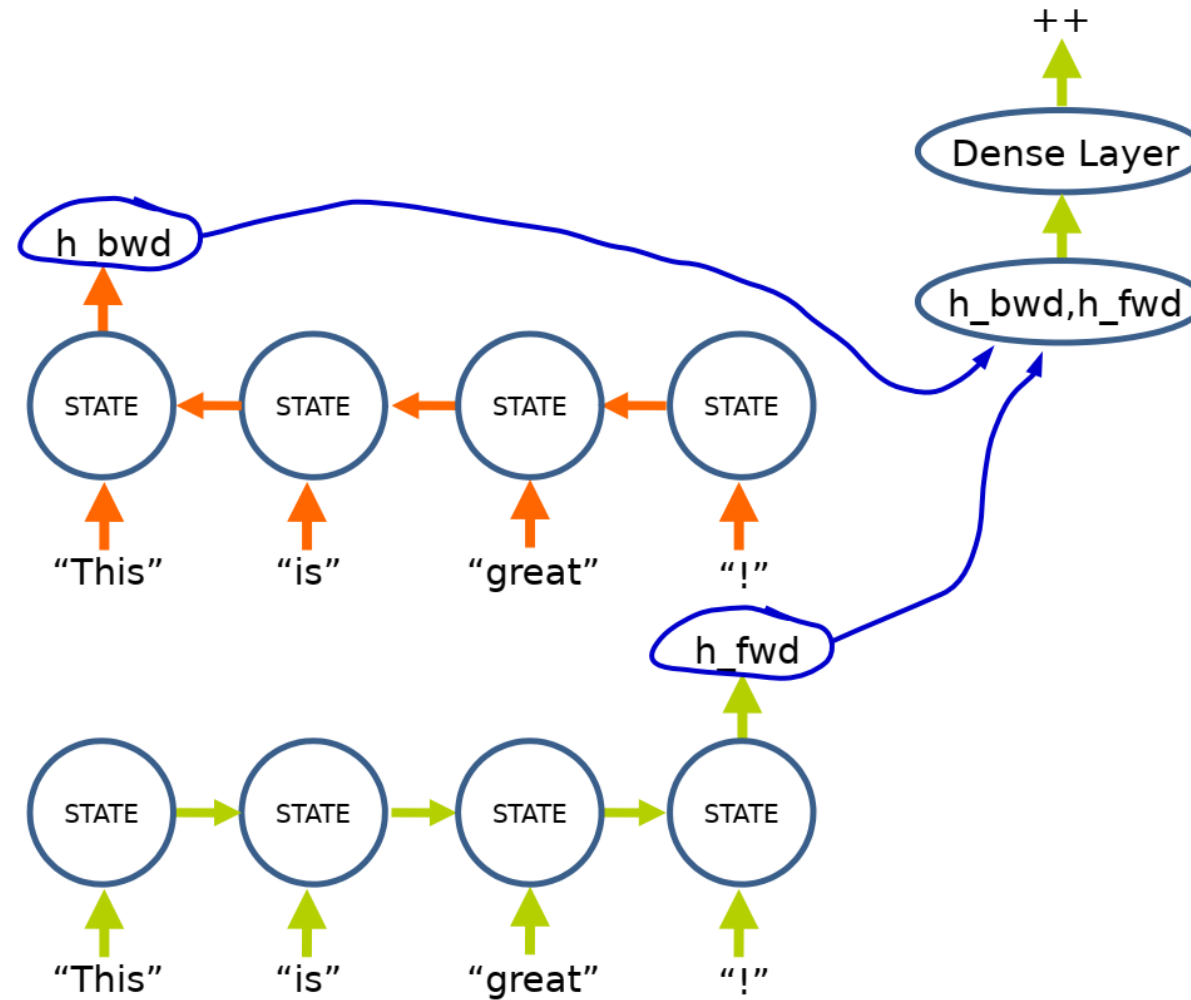


Conceptualicemos posibles aplicaciones de las RNN: predicción de sentimiento



- En cada paso, el estado oculto modela la historia/características de la secuencia hasta el momento.
- En **este problema particular**, podemos capturar más información de la secuencia, mediante una **RNN bidireccional**.

Conceptualicemos posibles aplicaciones de las RNN: predicción de sentimiento con **RNN bidireccional**



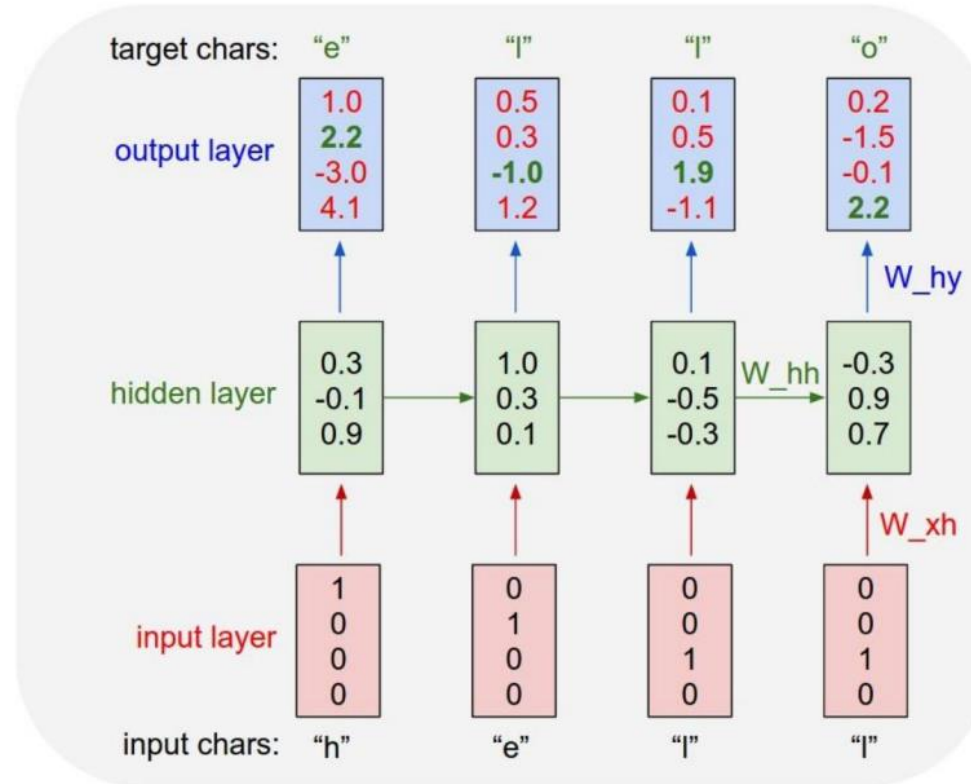
Vamos a Colab...



Conceptualicemos posibles aplicaciones de las RNN: generación de texto

Vocabulary:
[h,e,l,o]

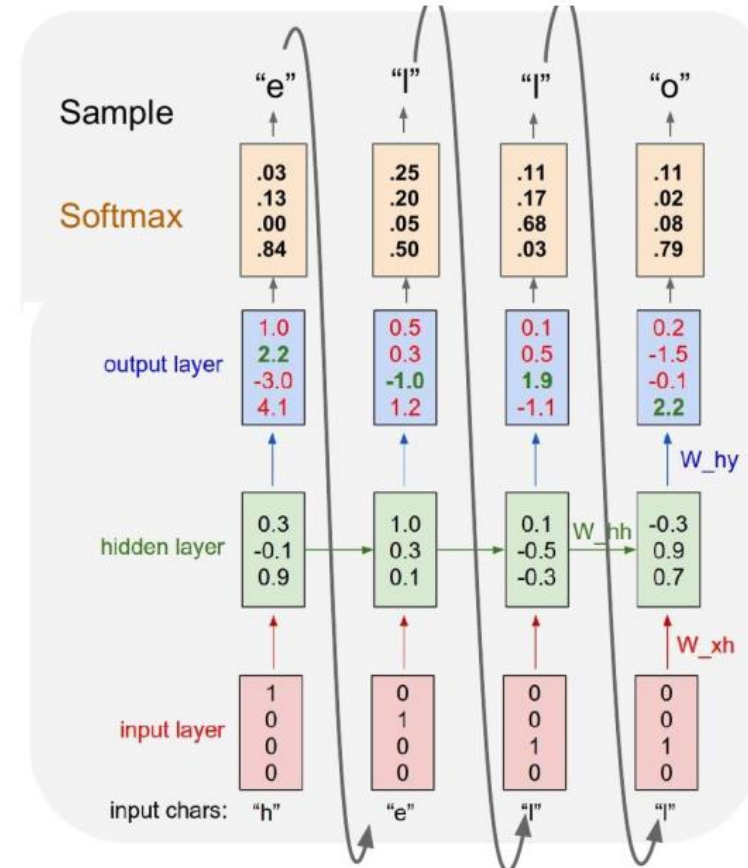
Example training
sequence:
“hello”



Conceptualicemos posibles aplicaciones de las RNN: generación de texto

Vocabulary:
[h,e,l,o]

At test-time sample
characters one at a time,
feed back to model



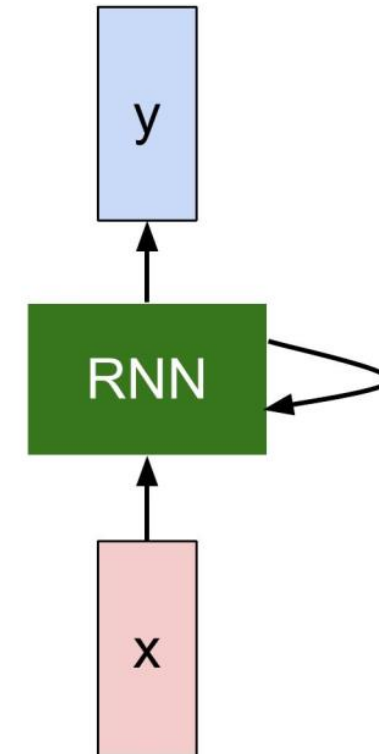
Conceptualicemos posibles aplicaciones de las RNN: generación de texto

THE SONNETS

by William Shakespeare

From fairest creatures we desire increase,
That thereby beauty's rose might never die,
But as the ripper should by time decease,
His tender heir might bear his memory:
But thou, contracted to thine own bright eyes,
Feed'st thy light's flame with self-substantial fuel,
Making a famine where abundance lies,
Thyself thy foe, to thy sweet self too cruel:
Thou that art now the world's fresh ornament,
And only herald to the gaudy spring,
Within thine own bud buriest thy content,
And tender churl mak'st waste in niggarding:
 Pity the world, or else this glutton be,
 To eat the world's due, by the grave and thee.

When forty winters shall besiege thy brow,
And dig deep trenches in thy beauty's field,
Thy youth's proud livery so gazed on now,
Will be a tatter'd weed of small worth held:
Then being asked, where all thy beauty lies,
Where all the treasure of thy lusty days;
To say, within thine own deep sunken eyes,
Were an all-eating shame, and thriftless praise.
How much more praise deserv'd thy beauty's use,
If thou couldst answer 'This fair child of mine
Shall sum my count, and make my old excuse,'
Proving his beauty by succession thine!
 This were to be new made when thou art old,
 And see thy blood warm when thou feel'st it cold.



Conceptualicemos posibles aplicaciones de las RNN: generación de texto

tyntd-iafhatawiaoihrdemot lytdws e ,tfti, astai f ogoh eoase rrranbyne 'nhthnee e
plia tklrqd t o idoe ns,smtt h ne etie h,hregtrs nigtike,aoaenns lng



train more

"Tmont thithey" fomesscerliund
Keushey. Thom here
sheulke, anmerenith ol sivh I lalterthend Bleipile shuw y fil on aseterlome
coaniogennc Phe lism thond hon at. MeiDimorotion in ther thize."



train more

Aftair fall unsuch that the hall for Prince Velzonski's that me of
her hearly, and behs to so arwage fiving were to it beloge, pavu say falling misfort
how, and Gogition is so overelical and ofter.



train more

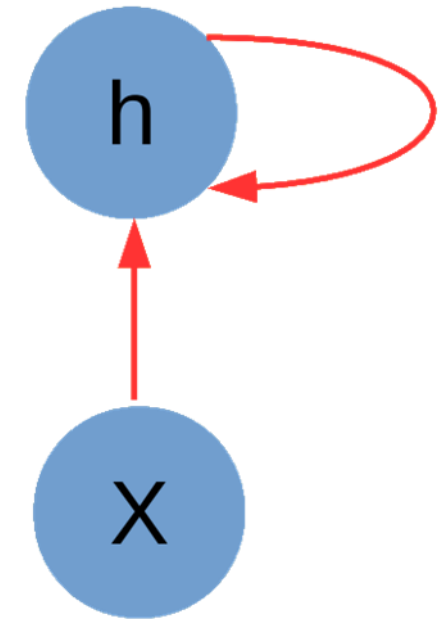
"Why do what that day," replied Natasha, and wishing to himself the fact the
princess, Princess Mary was easier, fed in had oftended him.
Pierre aking his soul came to the packs and drove up his father-in-law women.

Vamos a Colab...



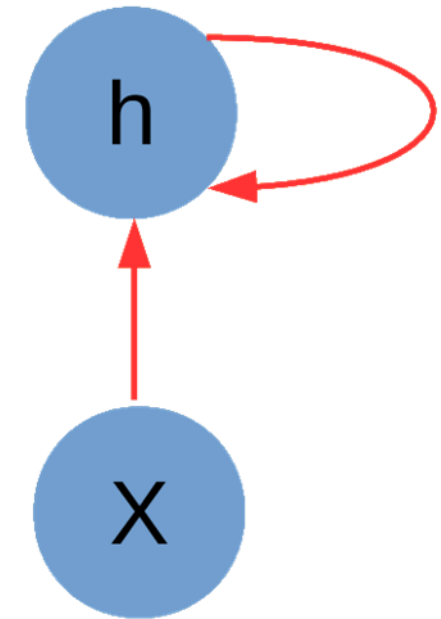
Vayamos ahora como entrenar una RNN

- Las operaciones en una RNN pueden descomponerse generalmente en tres bloques:
 1. Desde la entrada hasta el estado oculto W_{xh}
 2. Desde el estado oculto anterior al nuevo W_{hh}
 3. Desde el estado oculto a la salida W_{hy}
- El entrenamiento requiere lógicamente definir una función de pérdida adecuada a la tarea (cross-entropy, ranking, MSE, etc.).

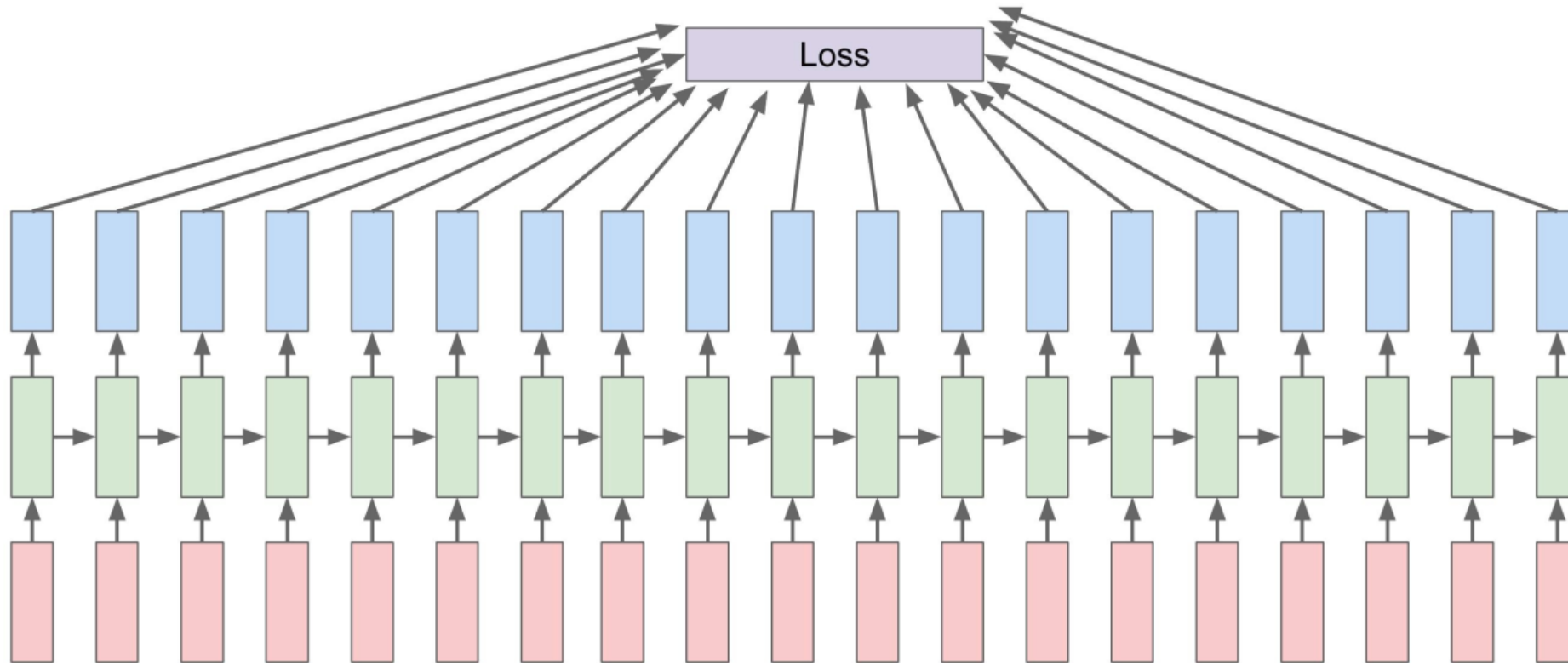


Vayamos ahora como entrenar una RNN

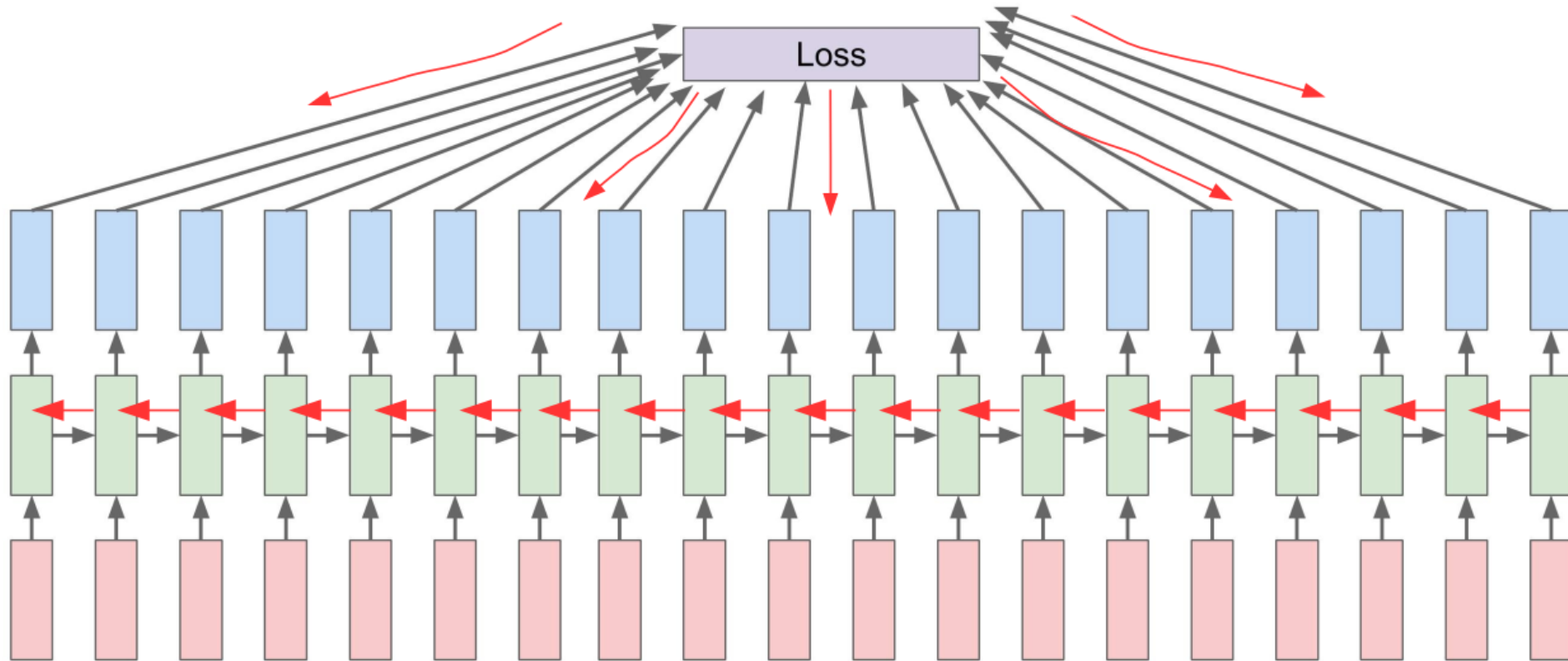
- Al igual que en las otras redes que hemos visto, la minimización de la función objetivo se hace con SGD.
- Dado que una RNN desenrollada es un grafo de cómputo, para calcular los gradientes basta aplicar *backpropagation* tal cual como antes.
- Esta técnica se conoce como *backpropagation through time* (BPTT).



Realizamos los cálculos “hacia adelante” a través de la secuencia completa, para calcular la pérdida



Luego propagamos la señal de error hacia atrás, a través de la secuencia, calculando y acumulando los gradientes (pesos son compartidos entre pasos t)



¿Cómo afecta la recurrencia al cálculo del gradiente?

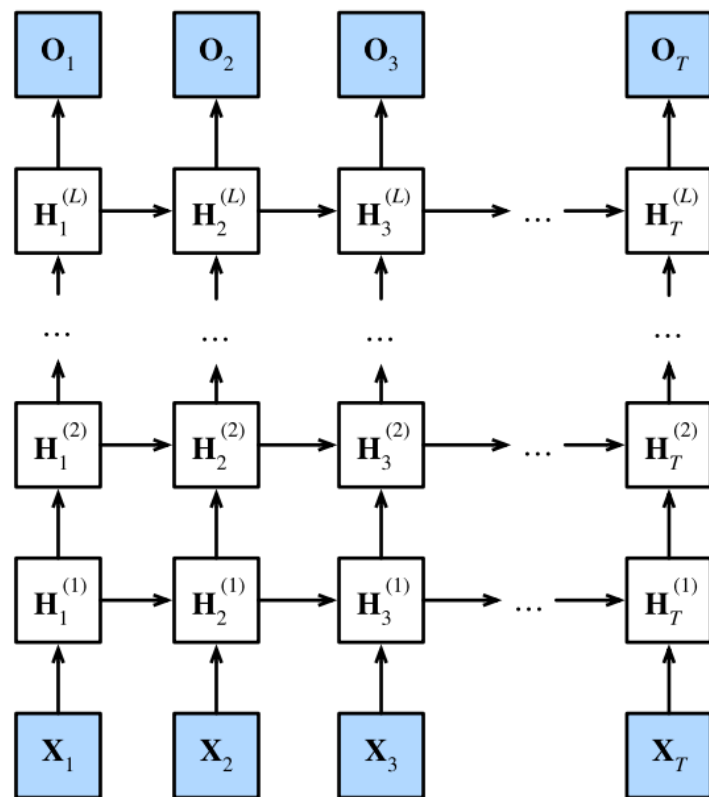
En términos simples, el cálculo del gradiente de complica bastante

$$h_t = f(x_t, h_{t-1}, w_h)$$
$$o_t = g(h_t, w_o),$$

$$L(x_1, \dots, x_T, y_1, \dots, y_T, w_h, w_o) = \frac{1}{T} \sum_{t=1}^T l(y_t, o_t)$$

$$\begin{aligned} \frac{\partial L}{\partial w_h} &= \frac{1}{T} \sum_{t=1}^T \frac{\partial l(y_t, o_t)}{\partial w_h} \\ &= \frac{1}{T} \sum_{t=1}^T \frac{\partial l(y_t, o_t)}{\partial o_t} \frac{\partial g(h_t, w_o)}{\partial h_t} \left(\frac{\partial h_t}{\partial w_h} \right) \end{aligned} \longrightarrow \frac{\partial h_t}{\partial w_h} = \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial w_h} + \sum_{i=1}^{t-1} \left(\prod_{j=i+1}^t \frac{\partial f(x_j, h_{j-1}, w_h)}{\partial h_{j-1}} \right) \frac{\partial f(x_i, h_{i-1}, w_h)}{\partial w_h}$$

Y si tenemos múltiples capas, esto se complica más aún.



Esta complejidad en el cálculo trae 2 problemas

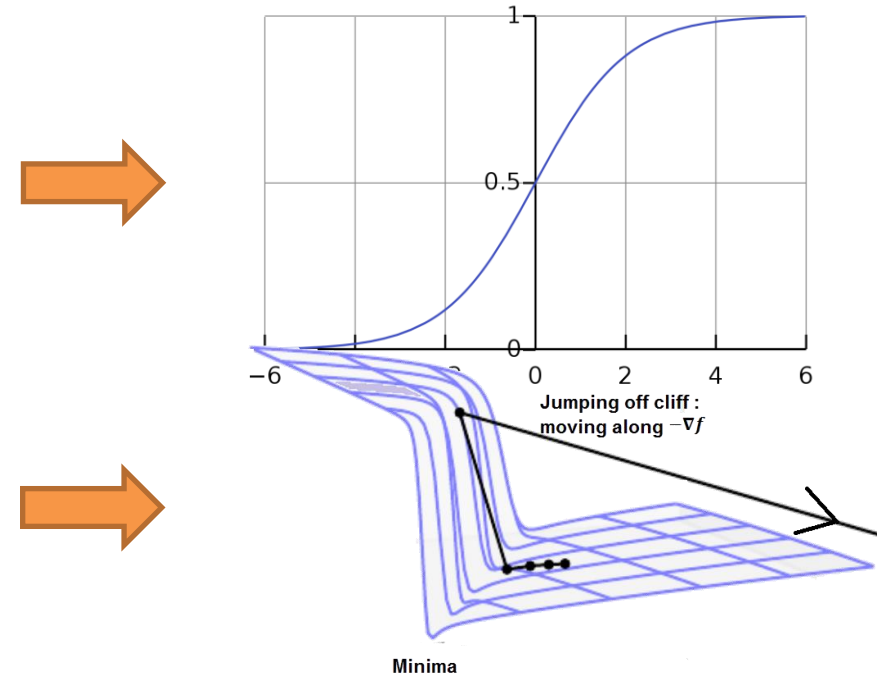
- Excesivos requerimientos computacionales
- Evolución del gradiente para secuencias largas:
vanishing y exploding gradient.

El primer problema son los requerimientos computacionales

- Dada la estructura recursiva, el cálculo del gradiente es excesivamente demandante en recursos, pero...
- ...una solución burdamente simple es truncarlo en cierto $t < T$ (*truncated backprop*).
- De esta forma, el modelo se centrará más en la influencia de las dependencias cercanas que las lejanas.
- En la práctica, eso funciona muy bien y además evita el sobreajuste (alta varianza por efecto mariposa), pero pierde capacidad de modelación para secuencias largas.

Un segundo problema con las secuencias largas son los *exploding & vanishing gradients*

- Regla de la cadena implica el producto repetido de sigmoides, lo que implica una reducción progresiva de la norma del gradiente (*vanishing gradient*)
- Podríamos entonces cambiar la función de activación por una ReLU, pero esto lleva un aumento progresivo de la norma del gradiente (*exploding gradient*).
- Es posible evitar esto si se trunca el gradiente, pero el rendimiento se ve afectado.
- Numéricamente, estos dos problemas son caracterizados por el valor singular (SVD) más grande de las matrices W (< 1 gradiente se desvanece, > 1 gradiente explota), lo que no deja mucho espacio de maniobra para esta arquitectura.



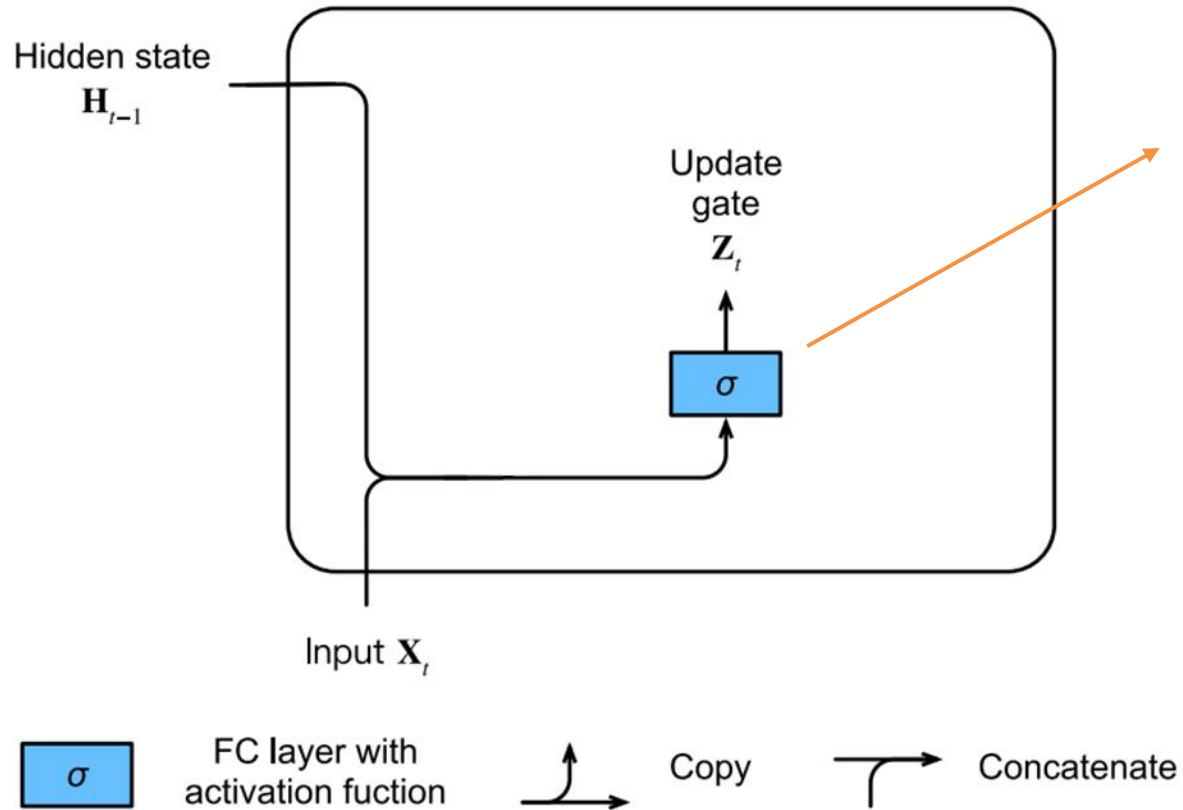
Revisitemos la regla de actualización para una RNN, para buscar una solución.

$$h_t = \sigma(W_{hh} h_{t-1} + W_{xh} x_t)$$

Podemos notar que para cada instante t , la memoria h_t es completamente recalculada, lo que se traduce en un flujo más “accidentado” del gradiente

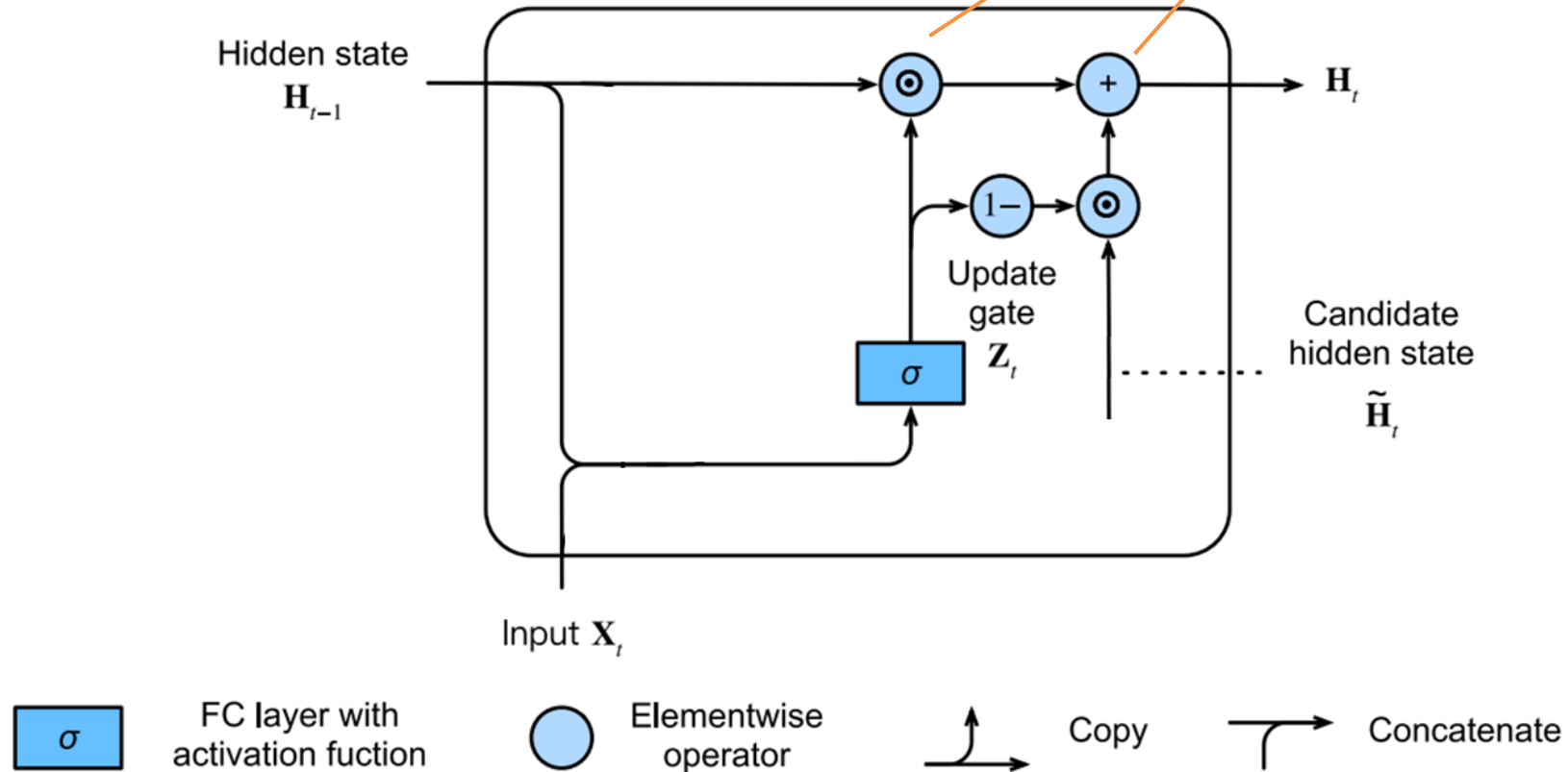
Para evitar esto, es necesario utilizar una arquitectura distinta. Veremos dos ejemplos:
Gated Recurrent Unit (GRU) y *Long short-term memory network* (LSTM).

Como elemento principal, una GRU considera una compuerta de actualización de la memoria

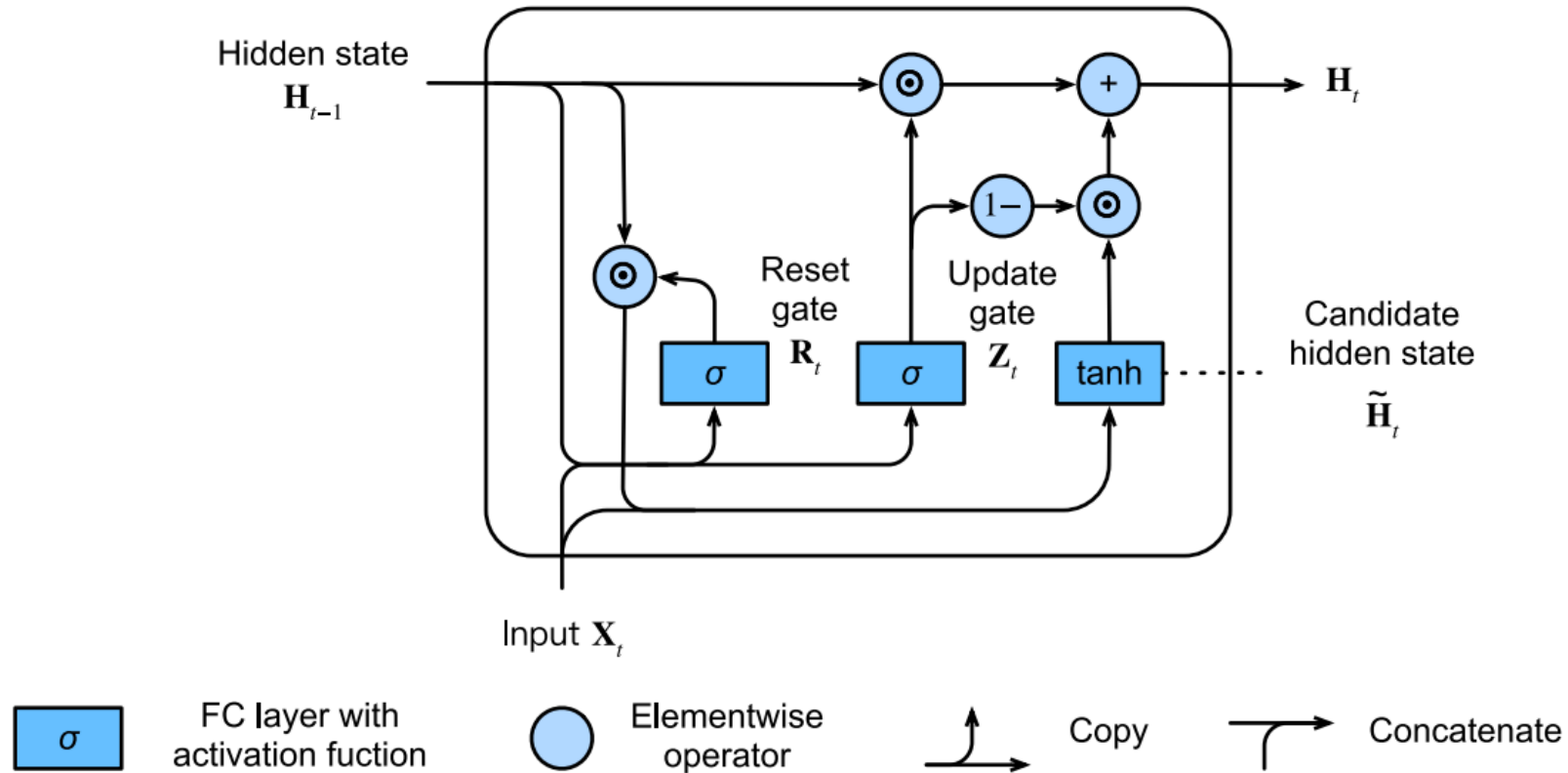


Esta compuerta controla cuanto se “olvida”
y cuanto se agrega a la memoria

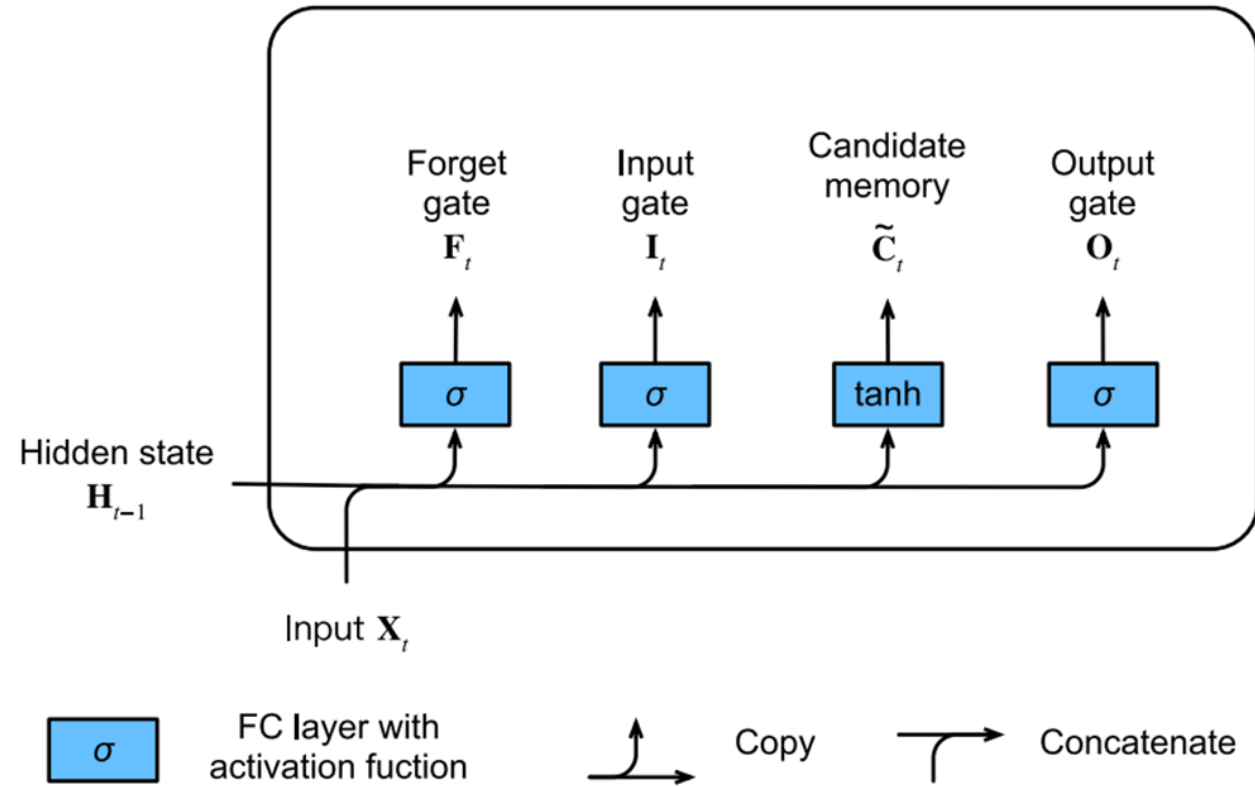
Acá se “borra” lo innecesario y luego
se rellena con información nueva



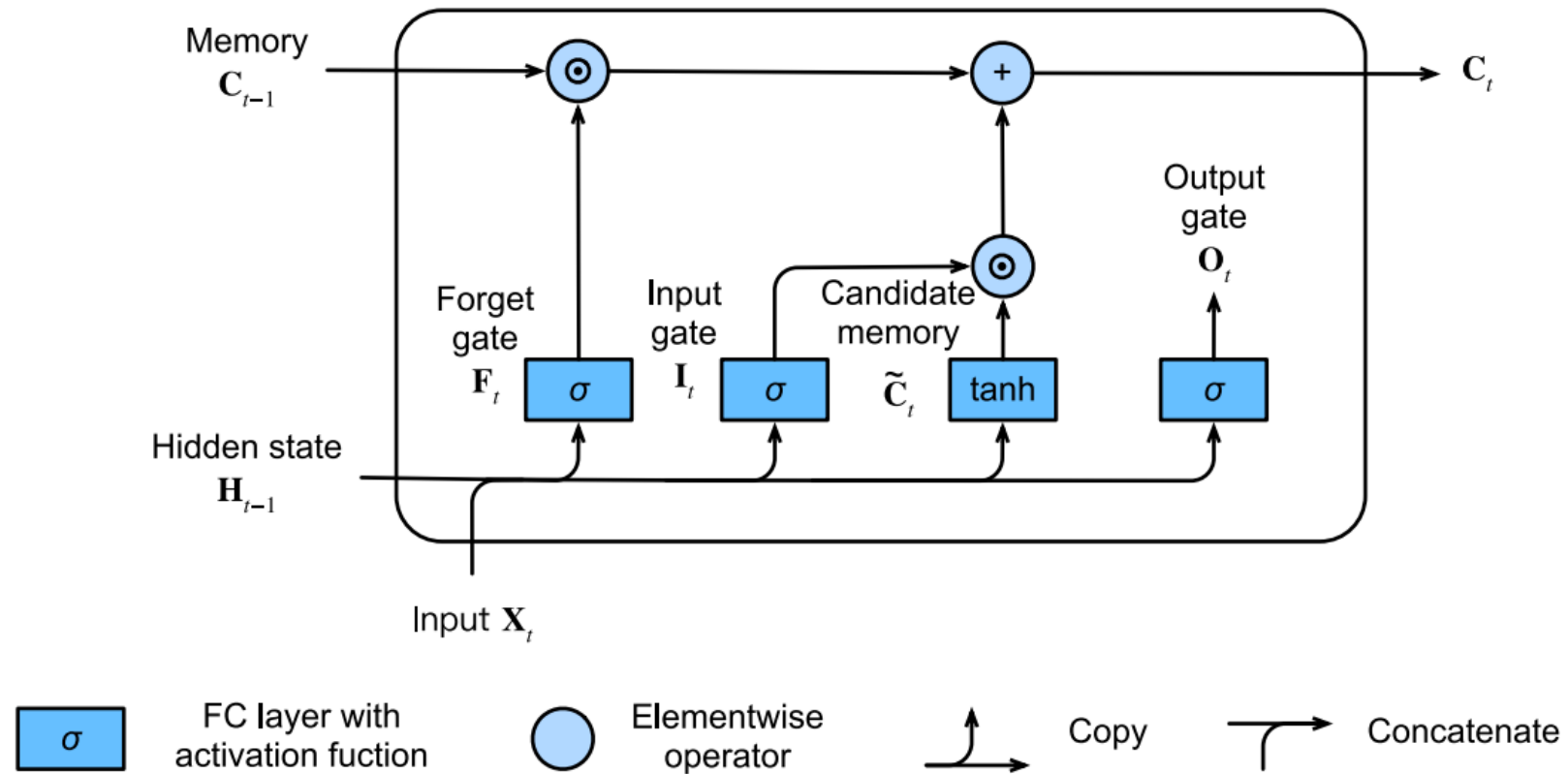
Lo que se “**agrega**” es a su vez modulado por una compuerta de *reset* (qué cosas de la memoria actual son necesarias para actualizar el estado)



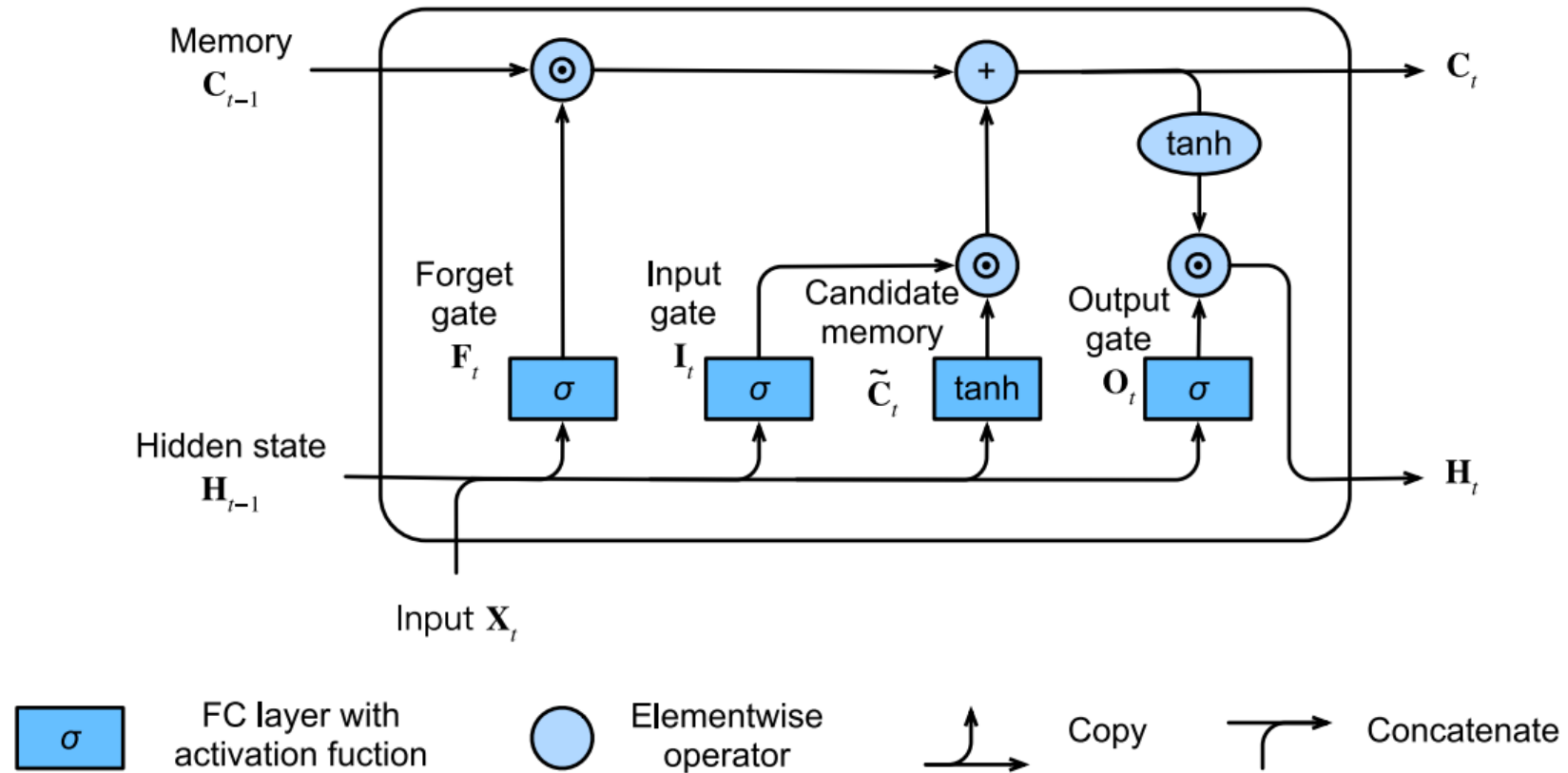
LSTMs extienden la idea de compuertas (pero son previas a GRU cronológicamente)



LSTMs separan explícitamente memoria de estado



Esto les permite tener un flujo del gradiente menos accidentado

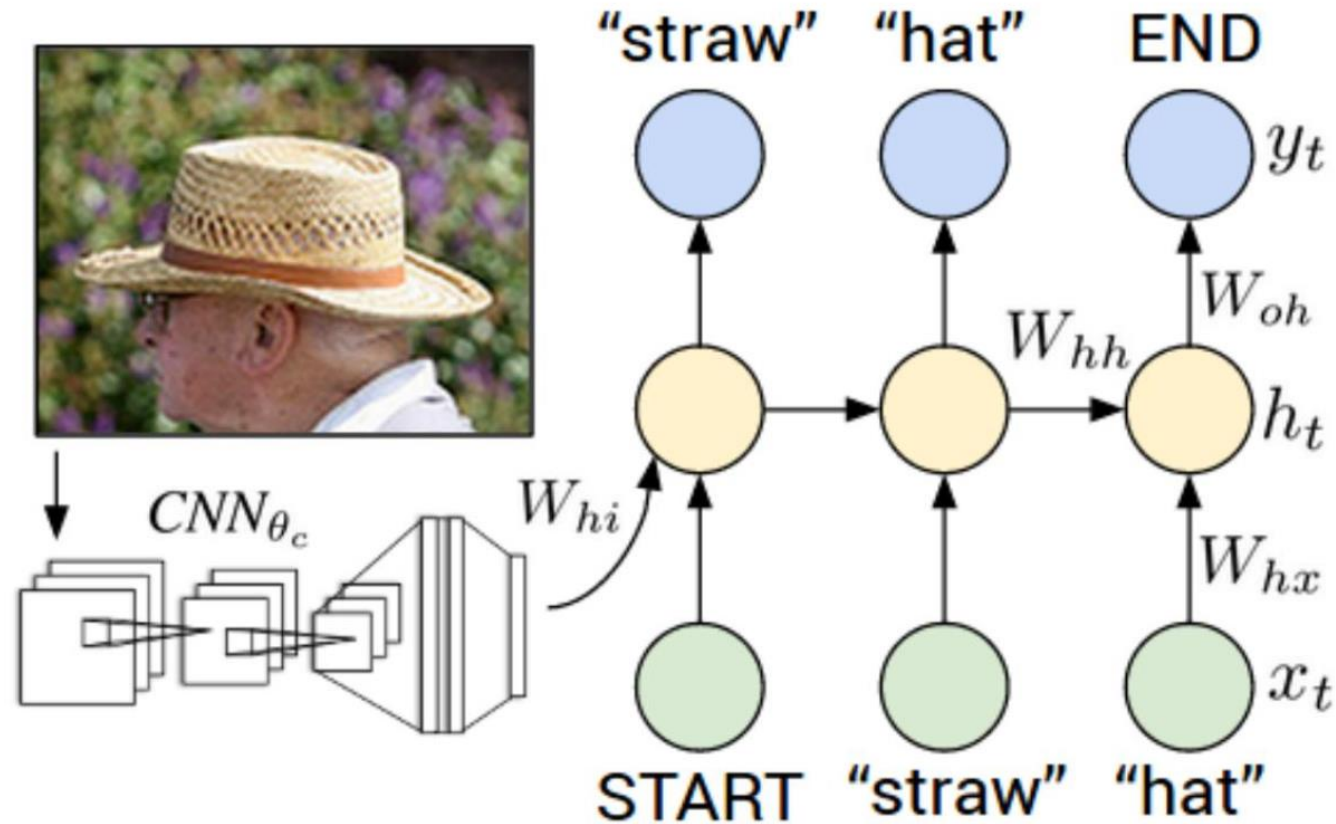


Cerremos con un caso de estudio

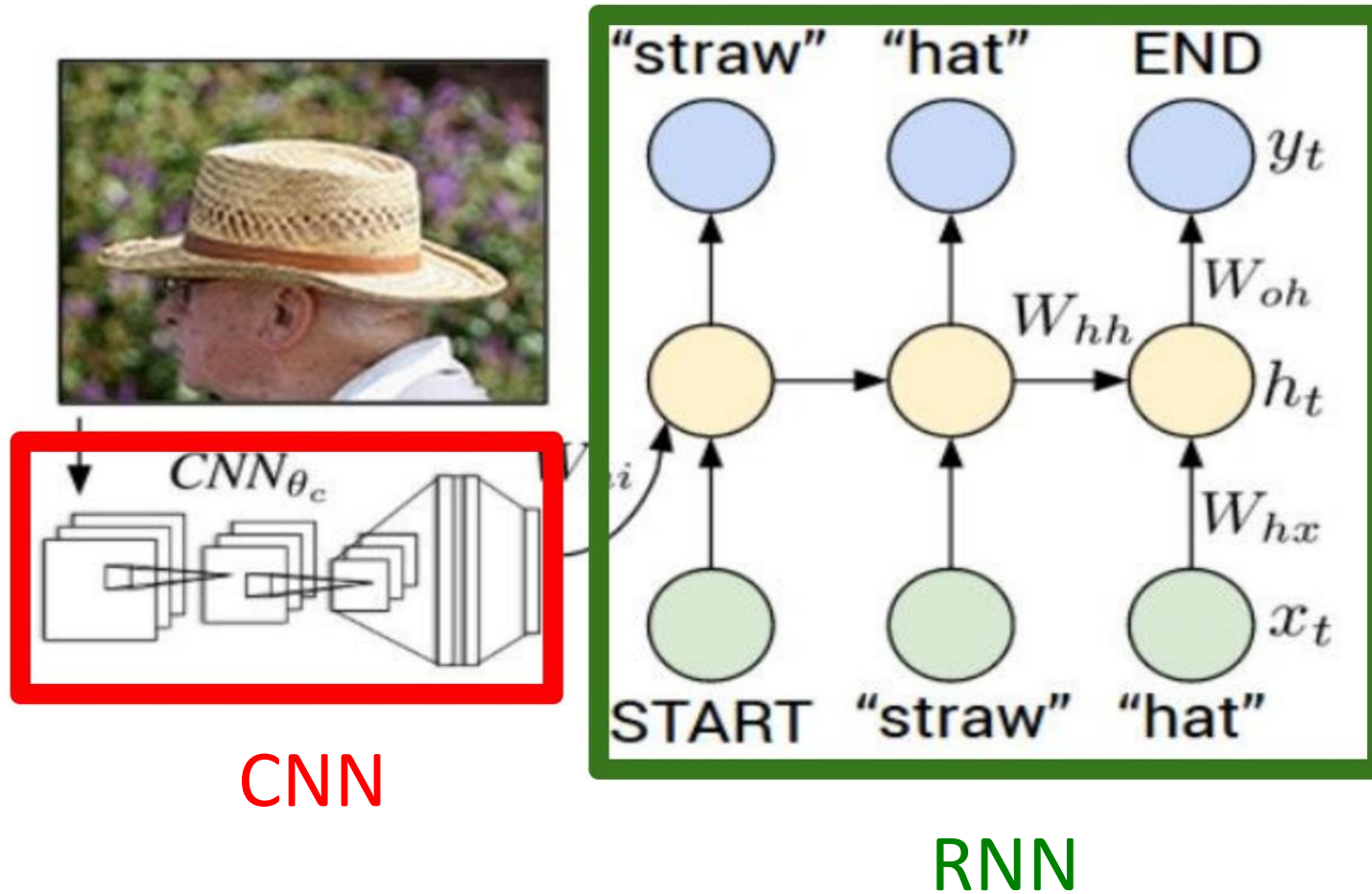
Automated Image Captioning with ConvNets and Recurrent Nets

Alex Karpathy, Fei-Fei Li (2015)

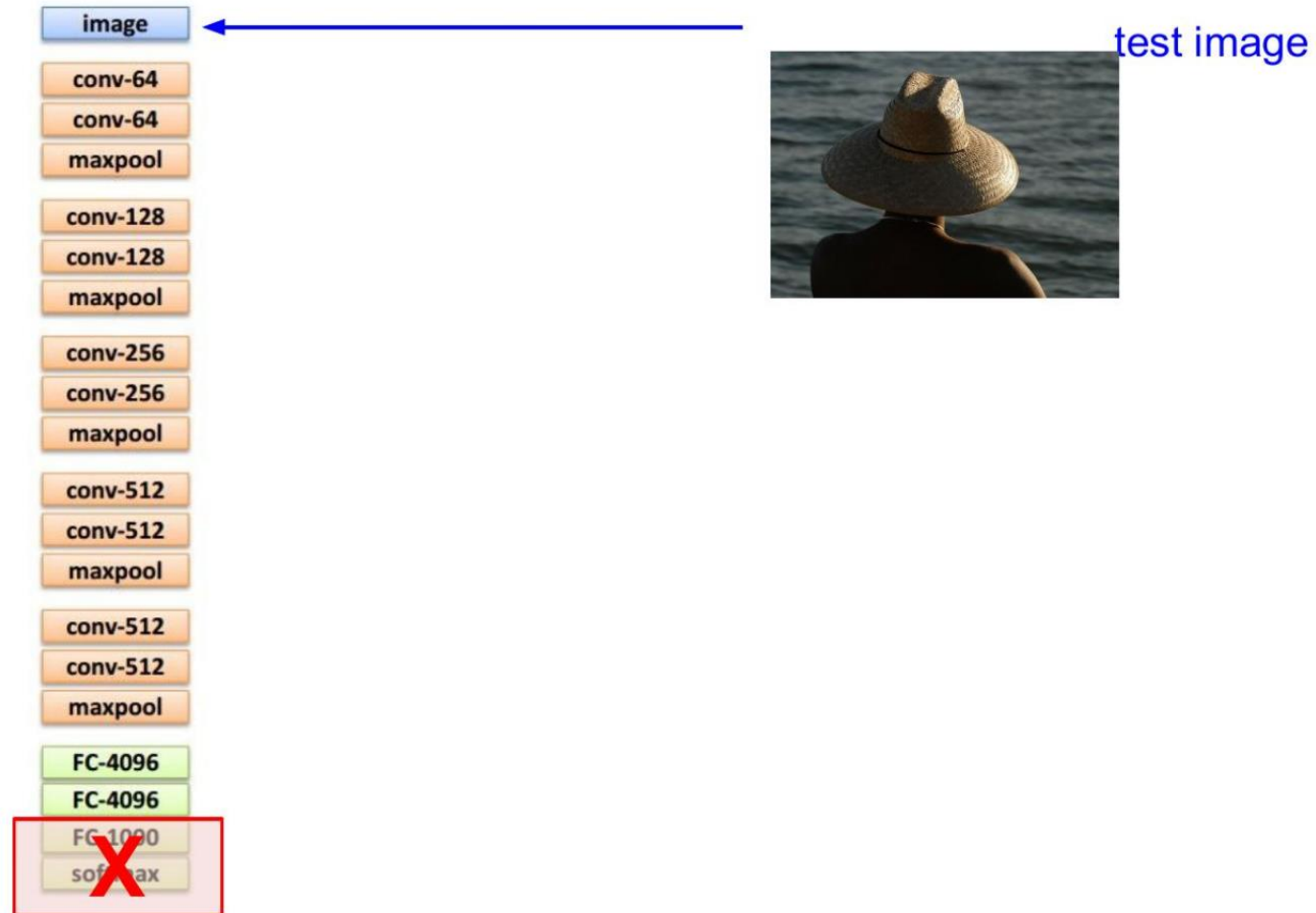
Podemos combinar CNN y RNN, aprovechando las fortalezas de cada una. Un ejemplo es *image captioning*



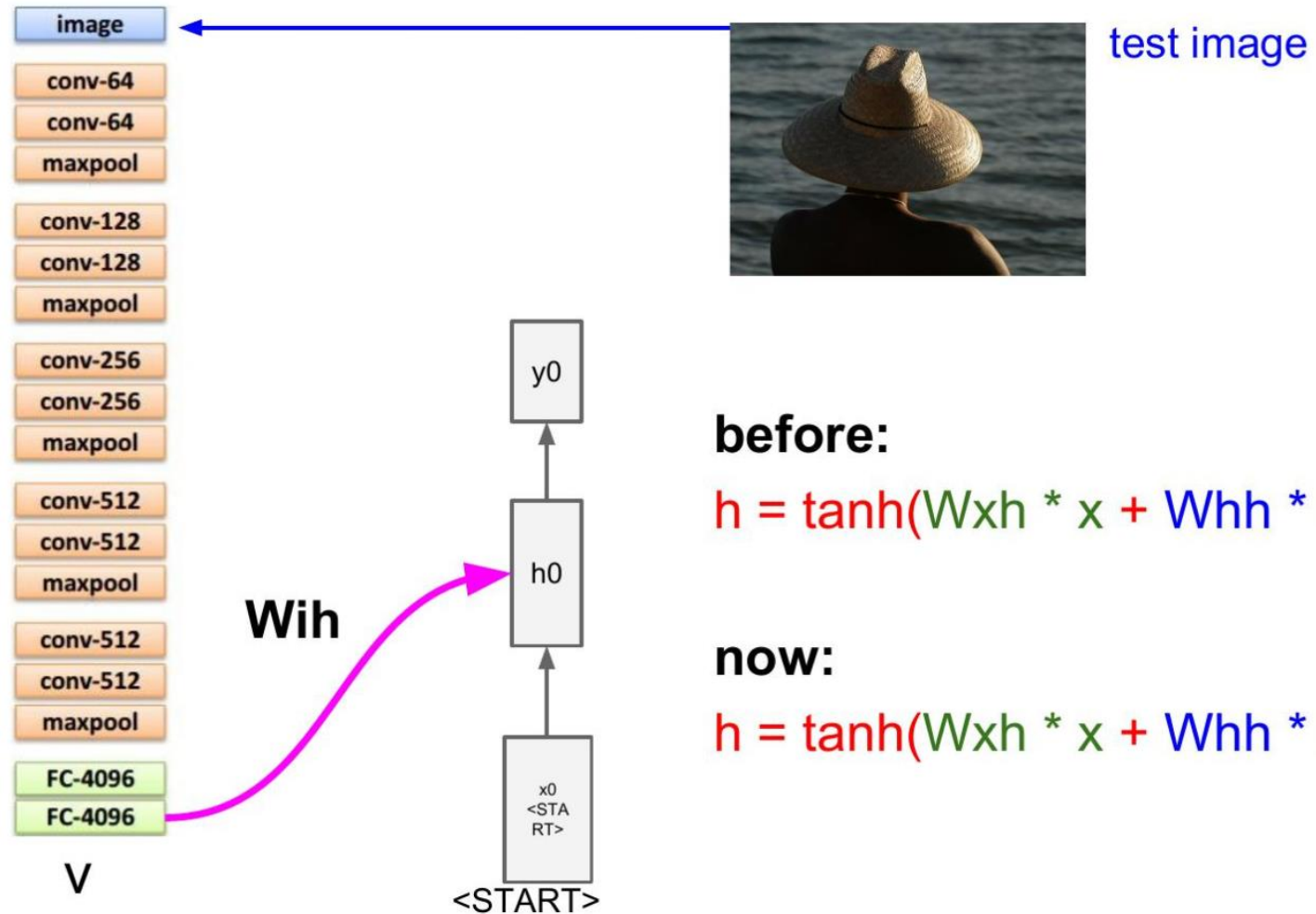
Podemos combinar CNN y RNN, aprovechando las fortalezas de cada una. Un ejemplo es *image captioning*



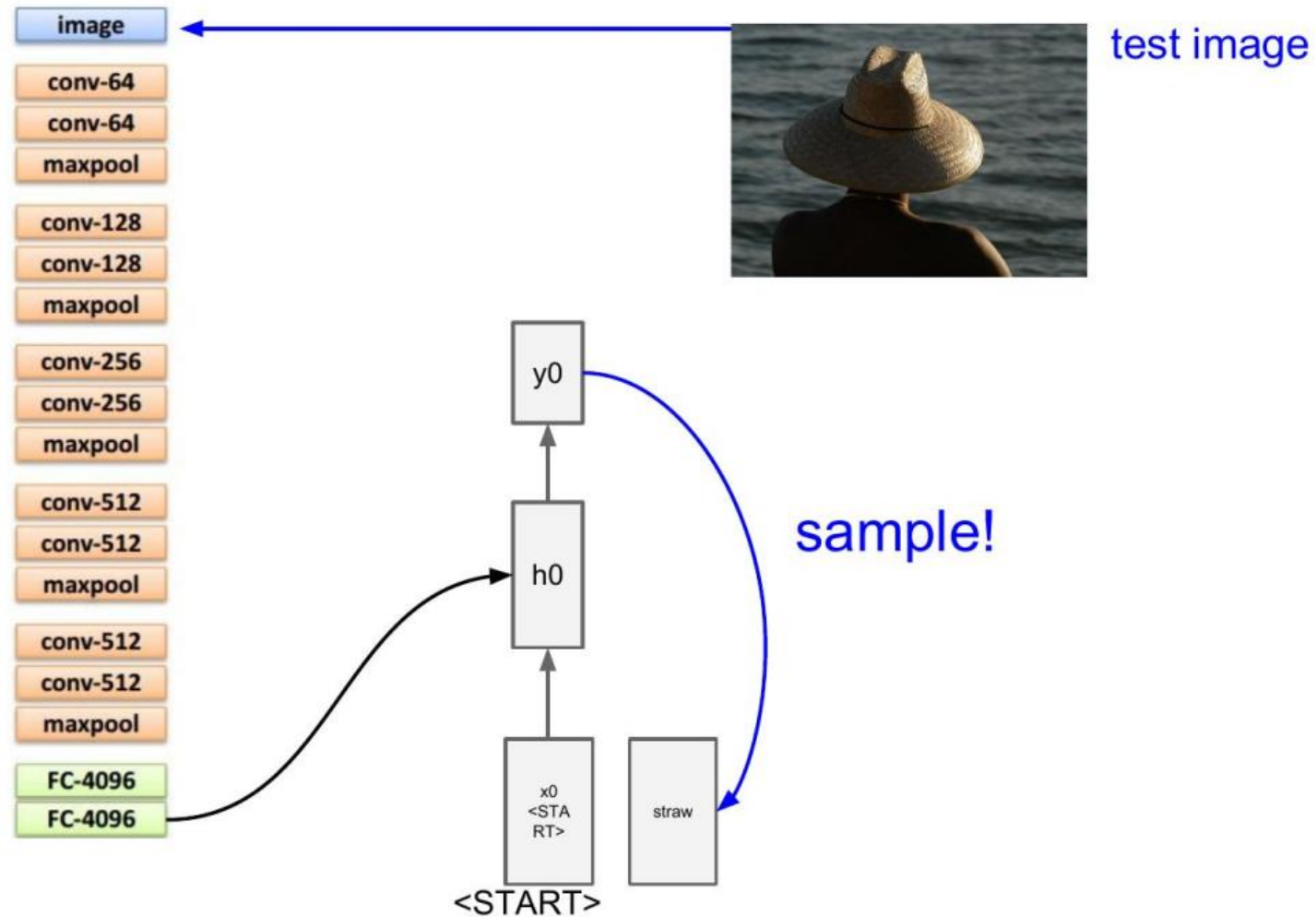
Podemos combinar CNN y RNN, aprovechando las fortalezas de cada una. Un ejemplo es *image captioning*



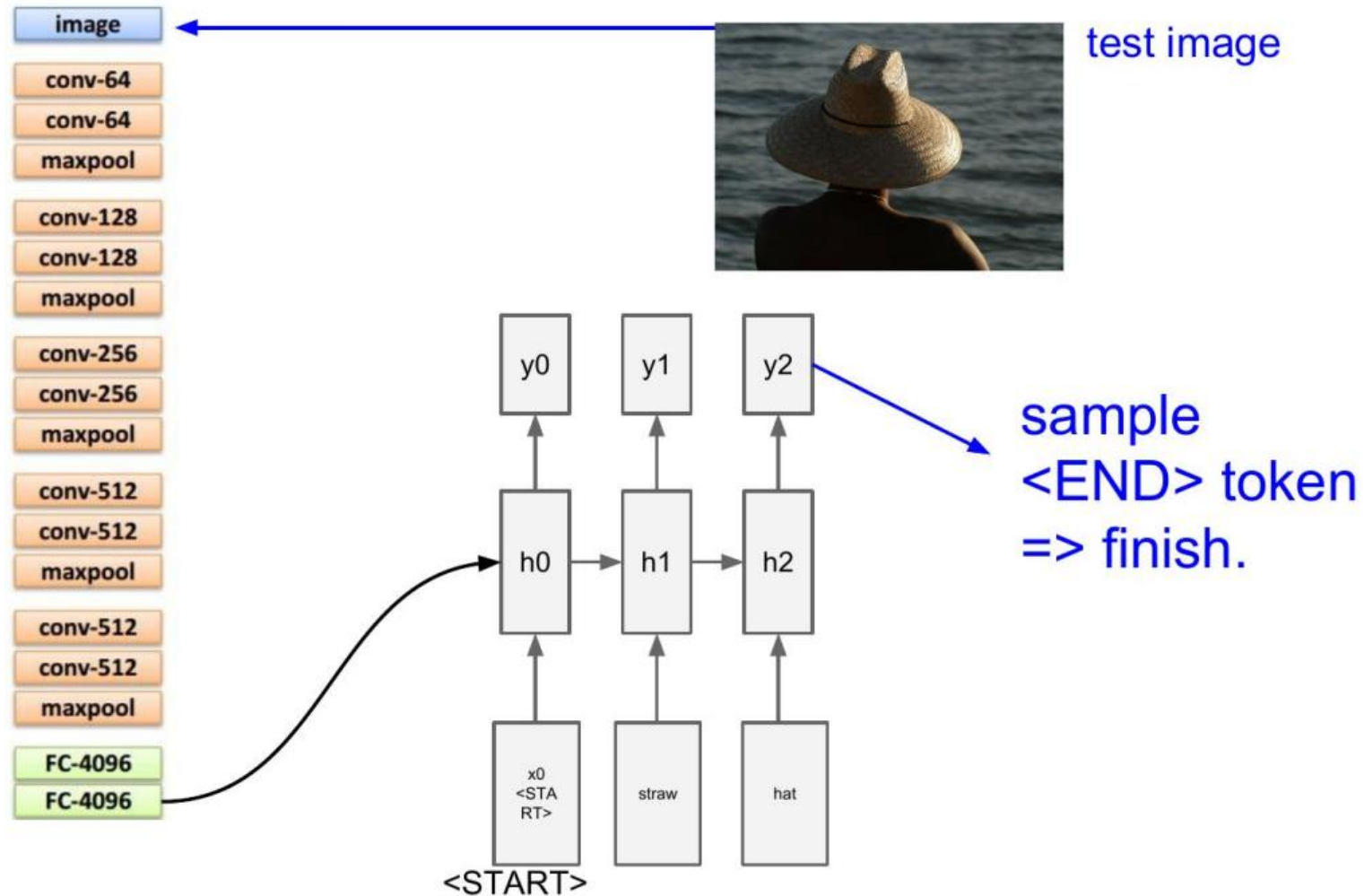
Podemos combinar CNN y RNN, aprovechando las fortalezas de cada una. Un ejemplo es *image captioning*



Podemos combinar CNN y RNN, aprovechando las fortalezas de cada una. Un ejemplo es *image captioning*



Podemos combinar CNN y RNN, aprovechando las fortalezas de cada una. Un ejemplo es *image captioning*



Podemos combinar CNN y RNN, aprovechando las fortalezas de cada una. Un ejemplo es *image captioning*



A cat sitting on a suitcase on the floor



A cat is sitting on a tree branch



A dog is running in the grass with a frisbee



Two people walking on the beach with surfboards



A tennis player in action on the court



Two giraffes standing in a grassy field

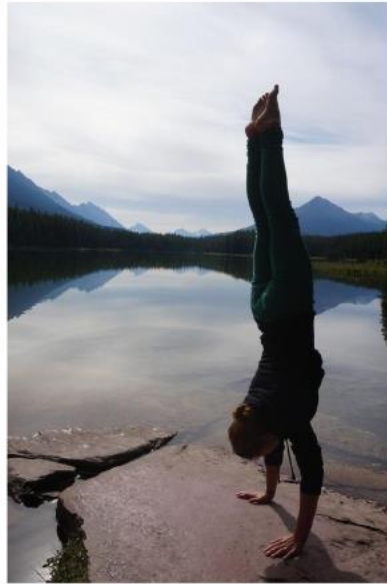
Podemos combinar CNN y RNN, aprovechando las fortalezas de cada una. Un ejemplo es *image captioning*



A woman is holding a cat in her hand



A person holding a computer mouse on a desk



A woman standing on a beach holding a surfboard



A bird is perched on a tree branch



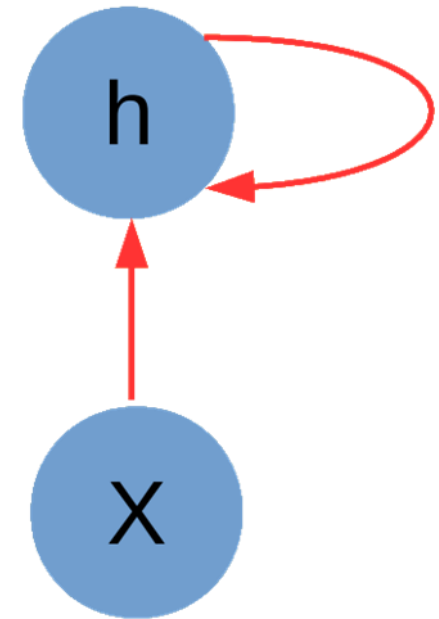
A man in a baseball uniform throwing a ball



<https://youtu.be/25jyl67-poQ>

Cuáles son los conceptos centrales de esta clase

- Redes recurrentes permiten modelar secuencias de largo arbitrario.
- El compartir parámetros de manera profunda a través de conexiones recurrentes les entrega gran poder.
- Alta flexibilidad y versatilidad les permite ser usadas en muchos dominios y acoplarse fácilmente con otras redes.
- Secuencias largas generan problemas en el flujo del gradiente. Arquitecturas GRU y LSTM permiten controlar esto.
- En la práctica LSTM y GRU son las arquitecturas más usadas.



Lecturas recomendadas

- Understanding LSTM Networks: <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>
- Exploring LSTMs: <http://blog.echen.me/2017/05/30/exploring-lstms/>
- Visualizing memorization in RNNs: <https://distill.pub/2019/memorization-in-rnns/>
- The Unreasonable Effectiveness of Recurrent Neural Networks: <http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación



Deep Learning

Redes Neuronales Recurrentes

Ariel Reyes
Dpto. Ciencia de la Computación